OptFuzz: Enhancing Compiler Testing via LLM-Powered Compilation Option Generation

YIFAN HUANG, YIBIAO YANG*, MAOLIN SUN, JIANGCHANG WU, QINGYANG LI, ZEYU LU, and YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, China

Compiler testing is crucial, as compilers serve as foundational infrastructure in software development. Effective compiler testing necessitates not only the generation of diverse test programs but also the systematic specification of compilation options. While existing research predominantly emphasizes test program diversification, it largely overlooks the strategic selection of compilation options required for comprehensive testing. Compilers typically offer a wide range of fine-grained compilation options that allow for precise control over the compilation process, resulting in a vast combination space. Exhaustive enumeration of all option combinations is computationally infeasible, while the stochastic generation of conflict-free, semantically meaningful options presents significant methodological challenges. To address these limitations, we propose OptFuzz, an innovative compiler testing framework that harnesses the generative capabilities of Large Language Models (LLMs) alongside the effectiveness of historical bug-triggering test programs for a comprehensive exploration of the compilation space. OptFuzz leverages LLMs to extract historical bug-triggering test programs from diverse bug reports, which are demonstrated to be effective in uncovering compiler bugs. Therefore, it overcomes the limitations of existing regex-based extraction techniques. Subsequently, OptFuzz employs a code abstraction extraction method based on intermediate representation (IR) to tackle the constraints of LLM context length limitations. Finally, the extracted IR is fed into LLM to generate effective compilation options for compiler testing. Through extensive experiments on GCC and LLVM, OptFuzz demonstrated superior bug detection capability compared to the random compilation space exploration method and other existing technologies. Notably, OptFuzz discovered 64 new bugs in GCC and LLVM, with 53 confirmed or fixed, highlighting our method's practical utility. The experimental outcomes also indicate that the IR-based analysis substantially decreases overhead and improves bug detection compared to direct utilization of source code.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Compiler fuzzing, large language model, compilation option

1 Introduction

Compilers are foundational infrastructure in software development, playing a pivotal role throughout the software engineering lifecycle. Virtually all software applications, from operating systems to sophisticated web browsers and even concise scripting tools, undergo transformation facilitated by compilers or similar translation mechanisms. However, compilers' inherent complexity makes them prone to errors. Bugs hidden within compilers can result in logically correct source code being compiled into erroneous binary code [1], thereby causing a chain of unexpected and potentially destructive behaviors. More seriously, a compiler defect may lead to exploitable

*Corresponding author.

Authors' Contact Information: Yifan Huang; Yibiao Yang, yangyibiao@nju.edu.cn; Maolin Sun; Jiangchang Wu; Qingyang Li; Zeyu Lu; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/5-ART

https://doi.org/10.1145/3817602

Just Accepted

software vulnerabilities [83, 84]. Consequently, ensuring the reliability and correctness of compilers is of critical importance.

In the last two decades, substantial research efforts have been devoted to compiler testing to improve compiler quality [4, 6, 7, 19, 32, 36, 41, 63, 82, 87]. In compiler testing, the creation of effective test inputs is of paramount importance. These inputs encompass not only test programs but also compilation options. Current research has introduced various techniques for generating test programs, which can be categorized into grammar-based and mutation-based approaches. Among the grammar-based testing methods, Csmith and YARPGen are prominent. Utilizing syntactic randomness, these tools generate a diverse array of test programs [45, 46, 87]. In contrast, mutation-based techniques create new test programs by mutating existing ones [10, 33, 36, 43, 44, 92]. These generated programs serve as inputs for stress-testing compilers, facilitating the identification of compilers bugs. Moreover, a recent study utilized test programs extracted from bug reports to test compilers, achieving notable results [94]. This study employs Support Vector Machine (SVM) algorithm for classifying code snippets, which are subsequently used in conjunction with differential testing for compiler validation.

Despite the fact that both the test program and the compilation option are indispensable inputs for compiler testing, the existing research in this domain has predominantly centered around the construction and generation of test programs. Concurrently, existing studies have infrequently delved into the generation of effective compilation options. While prior studies such as CTOS have taken important steps toward leveraging compilation options for compiler testing, effectively exploring the large and complex space of compilation options remains a challenging task [40, 91]. Compilers offer a wide array of compilation options, enabling precise control over the compilation process. Different compilation options can activate distinct compilation functionalities, leading to varying behaviors. These options may activate different passes or execute particular actions which could cause security issues stemming from optimizations [16, 83], and they constitute an essential part of compiler test inputs [7, 22, 48, 77]. This presents an opportunity to comprehensively explore the compilation space for rigorously testing compilers. These options, which comprise combinations of flags tailored to specific test programs, are essential for effectively exposing compiler bugs.

Bug reports contain valuable domain-specific knowledge for uncovering compiler bugs [70, 94], but automatically extracting test programs from them remains challenging [19, 94]. This is due to the prevalence of non-code content (e.g., comments, backtraces, patches) and the lack of precise, open-source extraction tools. Furthermore, compiler options are critical in triggering critical components of compilers in testing. However, constructing complex options remains challenging due to the interdependencies and potential conflicts among compilation flags, which may result in invalid tests. Both GCC and LLVM offer an extensive array of approximately 2000 fine-grained compilation options, which can be selected and combined, resulting in a vast combination space [22, 48]. As the number of potential option combinations increases exponentially, identifying the most effective set of fine-grained compilation options for a specific test program becomes increasingly complex [95]. Enumerating all possible combinations is not only impractical but also infeasible. Moreover, the random generation of effective and conflict-free compilation options presents significant challenges. Therefore, there is an urgent need for more effective strategies in the formulation of compilation options to facilitate comprehensive testing of compilers.

The advent of large language models (LLMs) has revolutionized various aspects of software testing [74, 76, 82, 85]. Innovations leveraging LLMs have been witnessed in areas like source code-based specification generation [86, 89], test case creation [3, 13, 14, 28, 61], fault localization [81] and program repair [30, 60, 79, 80]. The profound understanding of code rules and human language by pre-trained LLMs enables researchers to perform tasks that were previously unfeasible, paving the way for methodological innovations [20, 35]. Nevertheless, the substantial time and resource costs associated with LLMs necessitate careful consideration. Furthermore, integrating complex engineering code, which is rich in semantics and offers comprehensive testing of compilers, into LLMs, presents a challenge due to the constraints of their context length limitations, and this issue remains unresolved.

Just Accepted

Approach. Historical bug reports often include code-like content without standardized formatting, making it difficult to extract complete source code using simple regex or island parsing [19, 94]. Debug logs and patches frequently resemble real code, adding to the ambiguity. Additionally, the sheer number of compilation options and the numerous conflicts among them present considerable challenges in generating suitable options that can explore the compilation space effectively. To address these challenges, in this paper, we introduce a compiler testing framework that utilizes programs extracted from historical bug reports and employs LLMs to produce a variety of compilation options. We systematically retrieve resolved bug reports, download the corresponding attachments and online code links [57], and leverage the capabilities of LLMs to extract test programs from historical bug reports. To optimize the efficiency of LLMs' generation and enhance the generality of the method, we streamline the source code at the intermediate representation (IR) level. The selection of IR is based on the observation that once the source code is compiled into IR, the initially intricate statements and syntactic structures are transformed into more standardized and simplified forms. This involves extracting function definitions, control flow constructs and function invocations to create an abstracted version of the source program. Subsequently, we utilize LLMs to examine these abstracted source codes, facilitating the selection of compilation options that have a high probability of eliciting compiler errors. We evaluate the efficacy of our approach by applying it to two prominent C compilers, GCC and LLVM. Our method successfully uncovered 64 unique bugs, with 53 confirmed and 24 already fixed by developers.

Contributions. We make the following major contributions:

- **Novelty:** We introduce a novel approach that leverages Large Language Models (LLMs) to extract historical bug-triggering test programs from unstandardized issue reports in bug tracking systems. To address the content length limitation of code snippets for LLMs, we apply code abstraction at the Intermediate Representation (IR) level to create a uniform representation. Subsequently, we utilize LLMs to generate complex and effective combinations of compilation options tailored to the IR of specific historical test programs for comprehensive compiler testing.
- **Prototype:** We have implemented our proposed method in prototype form, named OptFuzz, demonstrating the feasibility and practicality of integrating complex compilation options with historical bug reports in compiler testing. This prototype serves as a valuable resource for further research and development in compiler testing methodologies.
- **Usefulness:** We conducted extensive evaluations of our proposed method using the two most widely used C/C++ compilers, GCC and LLVM. Experimental results shows that OptFuzz can generate highly diverse compilation option sequences for specific code snippets and has led to the discovery of 64 unique bugs across these mainstream compilers, demonstrating its effectiveness.
- **Benchmark:** We systematically mined and curated historical bug reports from both GCC and LLVM to create a benchmark dataset. This dataset captures a diverse array of historical bug-triggering test programs and has been shared with the research community to promote further investigation and advancements in compiler testing methodologies.

Outline. The structure of this paper is outlined as follows: Section 2 describes the background and our motivation. Section 3 presents our proposed method. Section 4 details our extensive evaluation. Section 5 further discusses the results of experiments. Section 6 reviews related work. Section 7 concludes the paper.

2 Background and Motivation

In this section, we provide a brief introduction to historical bug reports and compilation options, followed by a discussion on the application of LLMs in software testing. This leads to our motivation for using LLMs to extract historical unstandardized bug-triggering test programs from issue reports and generate complex compilation options for compiler testing.

Just Accepted

2.1 Historical Bug Reports

Bug report is a document or submission that describes a specific, reproducible issue or defect in a software system. Currently, GCC and LLVM[1] employ Bugzilla and GitHub issue tracking systems, respectively, as platforms for developers to document compiler bugs [21, 50].

To facilitate developers' better understanding and reproduction of reported bugs, users typically opt to upload the source code they input into the compiler to the platform. The methods of upload include direct attachment upload, sharing via links to online compilers, and incorporating the source code directly within the textual description of the bug report. However, our investigation in Table 2 has revealed that about 57% of bug reports lack attachments or URLs. Furthermore, even when attachments are included, there is a possibility that these attachments are not code files but rather items such as patches or output information. This necessitates the exploration of extracting code from the textual content of bug reports to enhance compiler testing.

It's challenging to collect programs in bug reports due to the fact that the issue reports are non-standardized. Both of GCC and LLVM bug management system lack a standardized reporting format, leading to ambiguities in the placement of code segments within the reports. Additionally, the debugging information and patch segments, which have a high degree of semantic similarity to the code, also pose significant challenges in extracting complete and effective code segments from historical issue reports.

2.2 Compilation Options

Compilation options are parameter settings for compilers when processing source code, influencing compilers' behavior and the generated target code. Modern compilers (e.g., GCC and Clang) use a series of compilation options to achieve various objectives, such as reducing compilation time, minimizing binary file size, and improving runtime performance. These options encompass numerous optimization techniques with complex interactions significantly impacting final compilation results [29]. The key challenge is finding optimal balance between code generation quality and maintaining expected program behavior, considering factors like execution speed, and resource utilization [27, 73].

GCC provides numerous compilation options categorized into different categories such as program compilation, diagnostic message options, warning options, static analysis, machine-specific, etc. Clang's options also fall into similar categories, although they differ slightly in specific options [22, 48]. GCC uses a strategy called pass to manage the complex compilation optimization process, which is divided into multiple passes, each performing a specific optimization operation [24]. The execution order of all passes is predefined, and each involves a complete scan or processing of the program. For example, the dead code elimination pass is implemented as `pass_dce`. Compilation options in GCC act as boolean variables and can be enabled or disabled with compilation options. The LLVM compilation process also uses passes, but the order can be affected by the user to some extent [56].

GCC and LLVM offer various compilation options, but work differently. GCC has a predefined optimization template, simplifying user choices but limiting flexibility. LLVM allows fully customizable optimization, providing more flexibility but requiring a deeper understanding of each option's purpose [54]. In addition to optimization options, compiler options encompass linking, memory checks, etc, which influence compiler's behavior in various ways. Research in compiler optimization focuses on identifying and addressing errors within compilers [7, 38, 45, 46, 77], highlighting the importance of thorough testing to uncover optimization errors.

2.3 Large Language Models

Large Language Models (LLMs) are deep learning models based on transformer architecture [75], capable of understanding and generating high-quality natural language text. By training on massive textual data extracted

[1]Note that LLVM issue reports were not on GitHub but on Bugzilla initially. They were later transferred to GitHub in recent years [47].

Just Accepted



Reporter 2013-09-07 04:31:23 UTC — Description

```
The following code causes an ICE when compiled with the current
gcc trunk ...

$ gcc-trunk -O1 -c small.c small.c: In function 'main':
small.c:20:5: internal compiler error: Floating point exception
int main ()
    ^ 0x924b2f crash_signal
            ../../gcc-trunk/gcc/toplev.c:335
      0x77ecd3 fold_array_ctor_reference
            ../../gcc-trunk/gcc/gimple-fold.c:2816
...
----------------------------------------
struct U {};
static struct U b[1] = { };
int a, **c, d, *e, f;
extern void bar (int, int, int, struct U);
extern void foobar (int, int, int);
extern int baz (int, int);

static void foo () {
  bar (d, 0, 0, b[0]);
  foobar (0 >= f, 0, 0);
  **c = 0 == a;
  baz (0, *e);
  *e = baz (0, 0);
}
int main () {
  foo ();
  return 0;
}
```

Developer1 2013-09-08 08:13:45 UTC — Comment1

```
...
2820  if (inner_offset + size > elt_size.to_uhwi () *
BITS_PER_UNIT)
(gdb) print elt_size
$1 = {low = 0, high = 0}
...
```

Developer2 2013-09-09 08:29:07 UTC — Comment2

```
I believe the following should be invalid C
struct U {};
static struct U b[1] = { };
...
```

Developer3 2014-01-17 10:15:32 UTC — Comment3

```
And to actually fix this bug, the following should
be enough...
--- a/gcc/gimple-fold.c
+++ b/gcc/gimple-fold.c
-       || elt_size.slt (tree_to_double_int (TYPE_SIZE_UNIT
(type))))
......
+       || elt_size.is_zero ())
      return NULL_TREE;
```

Fig. 1. The issue report of GCC #Issue 58346. The information provided by the reporter consists of source code and compiler error messages, whereas the developers' responses contain debugging details, minimized source code segments, and fragments of a patch implemented by the developers. It's evident that a significant portion of the textual information within the report is similar to source code, particularly in the patches and debugging outputs. These segments are semantically difficult to distinguish from program fragments, which poses significant challenges for traditional extraction techniques.

Just Accepted

from real world [2], LLMs have demonstrated impressive performance across various tasks in the field of natural language processing (NLP).

The Transformer model effectively captures contextual relationships and generates coherent outputs. For example, in code generation or code completion tasks, LLMs are trained on vast amounts of code data through unsupervised learning to grasp the syntax, structure, and common patterns of code [64]. By maximizing the accuracy of predicting next code tokens, the model gradually enhances its understanding and generation capabilities of code semantics. Recently, researchers have proposed employing well-designed prompting strategies or fine-tuning techniques for adapting LLMs to specific domain tasks, such as domain-specific code generation [18].

The emergence of LLMs has also enhanced software testing, including compiler testing [76, 85]. This includes innovations in various fields such as specification generation based on source code, test case generation and error localization. Titanfuzz proposed using LLMs to generate code snippets directly and synthesize Python test cases for deep learning libraries by masking function bodies or parameters based on output results [13]. FuzzGPT fine-tunes LLMs by collecting historical code segments that triggered errors in deep learning libraries and uses prompts to generate Python test cases [14]. Fuzz4all and KernelGPT adopt carefully designed prompts to guide LLMs in generating test code for specific software or kernel programs [82, 86]. LLMs have a high degree of automation, significantly reducing human intervention and increasing testing efficiency. Due to the high understanding of code and human language by pre-trained models, researchers can perform many tasks previously done by humans, which also provides possibilities for methodological innovation.

2.4 Motivation

Bug-triggering Program Collection. Programs from historical bug reports contain rich program semantics. It has been confirmed that they have the ability to detect new bugs and explore the compilation spac [19, 94]. In

addition, programs from bug reports have also been widely validated for effectiveness in other areas of software defect detection. For example, **JavaTailor** [93] synthesizes diverse test programs by weaving the ingredients extracted from JVM bug reports into seed programs. **HistFuzz** [70] proposes enumerating skeletons within historically triggered code snippets to generate comprehensive test cases for SMT solvers. **Orion** [65] utilizes bug reports to inform the creation of test inputs for deep learning libraries. Additionally, since they previously triggered compiler bugs, they effectively probe boundary behavior, enhancing bug detection. Furthermore, test programs for one compiler are potentially effective for others. For example, test programs from GCC's bug programs can also lead Clang to crash [52, 53], and vice versa.

Although using historical bug-triggering programs for compiler testing is valuable, there is still a lack of effective tools for extracting them. This is due to the complexity of the non-standard and non-uniformed bug reports, which pose difficulty for extracting programs through straightforward strategies such as regular expression matching. Figure 1 shows a bug report of GCC, where we only present the initial bug report and the comments from other developers. As illustrated, there are multiple code snippets including the initial and reduced programs by developers. Additionally, the debugging information, patch details, and backtrace information discussed by developers are similar to the code snippets. These complexities hinder the use of historical bug reports and make it challenging for traditional methods to extract useful code snippets. Some researchers have proposed using feature extraction and Support Vector Machines (SVM) for classifying and identifying code snippets [94]. However, accurately collecting targeted programs from bug reports with this approach remains challenging. Also, the relevant tool with dataset is not publicly available.

To fill the gap in the community, we propose leveraging LLMs to process bug reports and extract code snippets. Due to the rich corpus used during their pre-training, LLMs can effectively distinguish usable code snippets embedded in natural language and stack information, greatly improving the efficiency and accuracy of code snippet extraction. This enables the practical large-scale application of programs within bug reports for testing purposes.

```
register struct str *env asm("ebp");
extern int foo (int base) ;
extern int fun (int base)
{
  return foo (base);
}
```

Fig. 2. This program cannot trigger a compiler bug without complex compilation options. However, it caused a GCC bug #Issue 116042 when compiled with the compilation option of -O2 -fno-omit-frame-pointer -fno-stack-protector -fno-strict-aliasing -g3 -Wno-pointer-arith.

```
extern int b[];
void foo ()
{
  unsigned long v1, v2, v3;
  #pragma omp parallel for schedule(static, 32) collapse(3)
  for (v1 = 0; v1 < 20; v1 += 2)
    for(v2=9223372036854775807LL; v2>9223372036854775807LL; v2-=3)
      for (v3 = 10; v3 > 0; v3--)
        b[64]++;
}
```

Fig. 3. This program cannot trigger a compiler bug without complex compilation options. However, it caused a GCC bug #Issue 120564 when compiled with the compilation option of -O2 -fopenmp -Wno-omp-pthreads.

Just Accepted

```
auto test() {
  return __array_extent(int, );
}
```

Fig. 4. This program cannot trigger a compiler bug without complex compilation options. However, it caused a GCC bug #Issue 120558 when compiled with the compilation option of `-pipe -MMD -MP -MQ ./ -MT test.o -MF test.d`.

```
int a;
```

Fig. 5. This program cannot trigger a compiler bug without complex compilation options. However, it caused a Clang bug #Issue 147187 when compiled with the compilation option of `-Xclang -fsanitize-system-ignorelist=./optfuzz -fbounds-check -fno-sanitize-link-runtime -fsigned-char`.

```
int main() {
  int n = 1;
  auto h = [&]<int>() { n = 0; };
  auto m = [h]<int j>() { h.template operator()<j>(); };
  m.template operator()<0>();
  return n;
}
```

Fig. 6. This program cannot trigger a compiler bug without complex compilation options. However, it caused a Clang bug #Issue 143266 when compiled with the compilation option of `-std=c++11 -pedantic -Wall -Wextra -Wconversion`.

Table 1. Distribution of Test Programs by Size (Lines of Code) Before and After Abstraction

	0∼100	101∼1000	1001∼10000	>10000
Origin	66.46%	10.14%	9.45%	13.96%
After Abstraction	77.01%	15.64%	6.22%	1.12%

Compiler Input Dimensions. In addition to testing programs, compilation options are equally important for compiler testing as test input. Compiler options can control compilers' behavior during code compilation. Using different options for the same code can yield entirely different results. By controlling compiler options, various passes within the compiler can be toggled, exploring more behaviors of compilers. Additionally, some compilation options can cause normally functioning compilers to crash. This is because different options may trigger distinct behaviors of compilers, such as different checks or optimizations, which could expose compiler defects depending on the specific code, as shown in Figure 2, 3, 4, 5 and 6. This provides a perspective for using compiler options for compiler testing.

Several methods exist for generating suitable compiler options [7, 32, 77]. One approach is to enumerate all possible option combinations and test them to identify issues [6]. However, this has significant drawbacks due to the large number of compiler options, making exhaustive enumeration impractical. Another approach is to use random algorithms to generate a set of compiler options [5, 7, 77]. However, dependencies between certain options can result in invalid tests if random combinations are incorrect. Additionally, potential conflicts between compilation options could also impede the application of these two methods. Both enumeration and random algorithms require substantial resources and time, making testing costly. Other approaches such as COTest and JOpFuzzer [7, 32], use machine learning or some pre-defined rules to generate compilation options. However, they either have poor generalization performance or their applicability is limited to predefined rules *w.r.t.* specific systems, which has certain limitations.

To address this, we propose employing LLMs to understand code and generate effective compiler options for testing. LLMs can effectively understand program semantics and select compiler options based on the program

Just Accepted

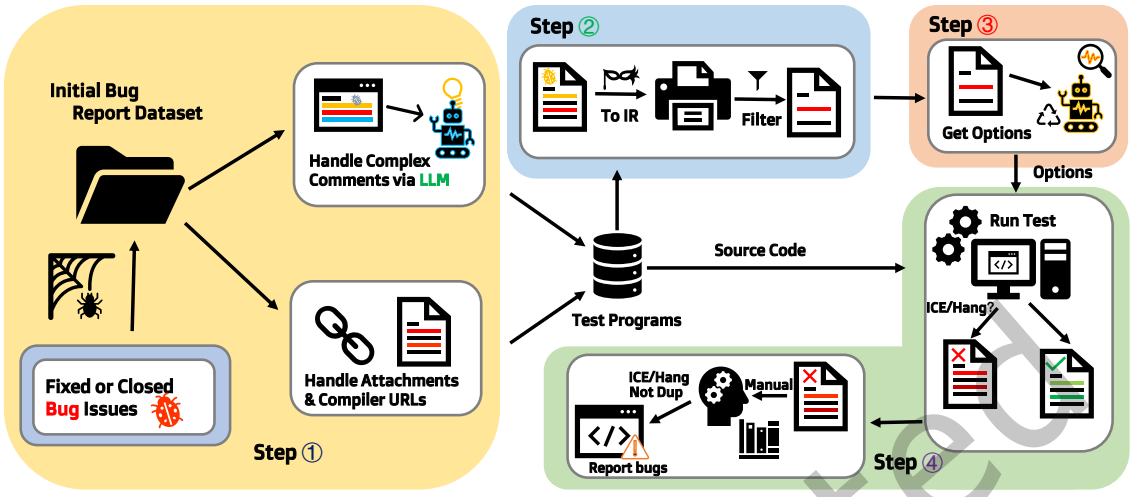


Fig. 7. Workflow of OptFuzz.

itself. The example in Figure 2, 3, 4, 5 and 6 adequately demonstrates this capability. However, employing LLMs directly in conjunction with bug-triggering program to generate complex compilation options poses challenges. As depicted in Table 1, a considerable proportion of bug-triggering programs exhibits remarkably extensive lengths, with over 20% of the test code surpassing 1,000 lines. This is because, although the developers encourage users to reduce their programs before reporting bugs, such as using tools like C-Reduce or Perses [63, 69], many users still upload unprocessed long code files. Some even upload pre-compiled code files, resulting in even more astonishing lengths. However, we are unable to use code reduction tools on these files since they require knowledge of the compiler issues (such as a compiler crash) triggered by the code, which is unknown before testing. Blindly reducing the code may undermine the effectiveness of the testing. Knowing that LLMs also have a good understanding of IR [11] , we have utilized the IR to abstract the source code, as illustrated in Table 1, thereby substantially reducing the relative length of lengthy code. We propose a compiler testing framework that integrates LLMs. This framework extracts programs from bug reports, abstracts code at IR level, and uses LLMs to generate complex compiler options for comprehensive compiler testing.

3 Approach

In this section, we first briefly introduce our framework and then explain each part in detail.

3.1 Overview

We propose a testing framework powered by LLMs in this study. The framework is mainly divided into four key steps: obtaining programs from historical bug reports, code abstraction, generating suitable compilation options for each program, and finally handling test results. Figure 7 depicts our proposed framework.

Specifically, step ① (In Algorithm 1 line 10-17) (Section 3.2) involves obtaining the test programs. Since programs from bug reports contain rich semantics, we choose to obtain test programs from them. To avoid duplication, OptFuzz filters and selects reports that have already been fixed. For reports with attachments, OptFuzz save and unzip the archives; for online links, it visit the links to obtain the code snippets. For reports

Just Accepted

without attachments and links, OptFuzz use LLMs to extract code snippets. Step ② (In Algorithm 1 line 18-21)(Section 3.3) extracts the key information of all programs. By plugin-based processing, we convert the code to IR, retaining important structures and simplifying complex or redundant parts to improve LLM's processing efficiency. In Step ③ (In Algorithm 1 line 22-24)(Section 3.4), we use Gemini [71] to generate complex compilation options based on the abstracted code snippets. Step ④ (In Algorithm 1 line 25-29)(Section 3.5) comprehensively evaluates the compiler's behavior, including triggering internal compiler error (ICE) and Hang issues. We analyze the results and then report potential bugs after reduction and deduplication. The main part of Algorithm 1 shows the whole process.

3.2 Historical bug-triggering Programs Acquisition

The first step is obtaining historical bug-triggering programs. Specifically, OptFuzz filters bug reports in GCC and LLVM, and to avoid duplication, it selects bug reports that have already been fixed. We implemented an HTML crawler to collect all filtered bug reports. As shown in Algorithm 1, the collected reports are classified into three categories: those with attachments (e.g., GCC #Issue101767) , those with online compiler links [57] in the report (e.g., GCC #Issue106687) , and those only have comments (e.g., GCC #Issue104007). For the reports with attachments, OptFuzz first save the attachments and then decompress them if they are compressed files. For the reports with online links, OptFuzz first save the links and then visit the webpages to obtain the code. Handling reports without any attachments or links is the most challenging. Code snippets are embedded in descriptions, patch code snippets, stack information, and other unrelated fragments, making manual classification costly. Using LLMs can achieve results similar to manual classification. For these reports, OptFuzz first record all the text as-is and input it into LLMs with a prompt to extract code snippets.

Prompt for getting code snippets in bug reports

> Please extract all possible complete C/C++ code snippets from this bug report comments (may exclude patches, which are generally incomplete or start with + / -), and output them to me in the order they appear. Thank you! Additionally, please ensure that no any other non-C/C++ code snippets are included in the output.

By leveraging LLMs, OptFuzz can effectively identify and extract valuable programs from bug reports. It is worth noting that OptFuzz does not perform any automatic repair or modification on the extracted code snippets. This design choice preserves the original bug-triggering programs reported by users and developers. Although some extracted snippets may not be compilable, such programs may still trigger bugs in compilers. In practice, both valid and invalid programs may trigger compiler bugs. Based on bug labels collected up to March 13, 2026[2], invalid programs account for a substantial fraction of compiler crash bugs: 29.12% for GCC ICE bugs and 51.13% for LLVM crash bugs.

3.3 Code Abstraction

Based on the empirical study [68], the results show that 95% of the test cases are smaller than 100 lines, 80% having fewer than 45 lines of code and more than 60% are smaller than 25 lines of code. However, the test cases analyzed in this study were derived from the test suites of GCC and LLVM [26, 55, 68], rather than actual code snippets from historical bug reports. While the programs in test suites also trace back to historical bug reports, they have typically undergone reduction by developers, substantially shortening their length.

[2]Bug labels are obtained from https://gcc.gnu.org/bugzilla/describekeywords.cgi and https://github.com/llvm/llvm-project/labels.

Just Accepted

Algorithm 1: The process of OptFuzz

Input: *Issues*: Total number of issues ; *C*: The compiler ; *LLM*: Specific LLM
Output: Reported bugs

```
1  begin
2      foreach Issue_i ∈ [1, Issues] do
3          P_set ← GetCode(Issue_i, LLM);
4      foreach p ∈ P_set do
5          IR_Abstraction ← CodeAbstraction (p);
6          Opts ← GetOption(IR_Abstraction, LLM);
7          foreach Opt ∈ Opts do
8              Report_Raw ← Test(C, Code_Raw, Opt);
9              BugReport ← Filter (Report_Raw);
10 Function GetCode(Issue_i, LLM):
11     if Issue_i has Compiler Explorer URL then
12         P_set ← Save URL data;
13     else if Issue_i has attachments then
14         P_set ← Download and save attachment;
15     else
16         P_set ← LLM_Issue_i;
17     return P_set
18 Function CodeAbstraction(p):
19     IR ← PreCompile(p);
20     IR_Abstraction ← FeatureExtraction(IR);
21     return IR_Abstraction
22 Function GetOption(IR_Code, LLM):
23     Opts ← LLM(IR_Code + Prompt);
24     return Opts
25 Function Filter(Report_Raw):
26     foreach r ∈ Report_Raw do
27         if (r is ICE or Hang) and r is not duplicate then
28             BugReport ← r;
29     return BugReport
```

Just Accepted

As detailed in Section 2.4, numerous programs found within bug reports are sourced from actual projects, many of which are significantly lengthy. Specifically, over 20% of these test codes exceed 1,000 lines. This considerable length persists despite developers encouraging users to minimize their programs before submitting bug reports, utilizing tools such as C-Reduce or Perses [63, 69]. Nevertheless, a substantial number of users continue to submit unaltered, lengthy code files, with some even providing pre-compiled files that contribute to this issue. Unfortunately, using code reduction tools on these programs is not feasible, as these tools require an

Algorithm 2: Control-Flow Abstraction via GCC Plugin

Input: $\mathcal{P}$, the programs with to be abstracted.
Output: $\mathcal{IR}_{Abstraction}$, the programs after abstraction.

```
1  begin
2      foreach Function F in P do
3          foreach Basic Block BB in F do
4              foreach GIMPLE statement stmt in BB do
5                  Identify GIMPLE code type T of stmt;
6                  if T ∈ {GIMPLE_CALL, GIMPLE_COND, GIMPLE_GOTO, GIMPLE_RETURN, GIMPLE_SWITCH,
                    GIMPLE_LABEL} then
7                      Use print_gimple_stmt to log stmt to IR_Abstraction;
8                  else
9                      Discard stmt (e.g., GIMPLE_ASSIGN, GIMPLE_NOP, etc.);
```

understanding of the specific compiler problems (e.g., compiler crashes) caused by the code before testing, which remain unidentified prior to testing.

Due to the context-length limitations of LLMs [58], using complex real-world programs directly may hinder the generation of effective compilation options. However, discarding such programs is suboptimal, as they contain rich semantics essential for compiler testing [41, 94]. To maintain semantic structure while ensuring LLM efficiency, we abstract the extracted programs.

We convert the code into its IR form and extract its control-flow skeleton. GIMPLE simplifies complex syntactic constructs into basic blocks of regularized statements. Statement types such as `GIMPLE_CALL`, `GIMPLE_COND`, `GIMPLE_GOTO`, and `GIMPLE_RETURN` preserve essential control-flow or invocation semantics, making them suitable for LLM processing. Algorithm 2 shows the main process of code abstraction.

More specifically, taking GCC's GIMPLE [23] and LLVM IR [11] as representative examples. GCC's IR (GIMPLE) maintains similarity with source code while offering more analyzability. This is achieved through its transformation from program control flow graph into distinct basic blocks, where each block contains sequential GIMPLE statements. Every statement is categorized by its GIMPLE code type[3], such as: `GIMPLE_ASSIGN` for assignment statements, `GIMPLE_CALL` for function calls, `GIMPLE_COND` for conditional jumps, `GIMPLE_GOTO` for unconditional jumps and `GIMPLE_RETURN` for returning from functions. LLVM IR employs a lower-level representation, where statement types are determined by the presence of specific opcodes such as: `br` for branch instructions (both conditional and unconditional), `call` for function calls and `ret` function returns.

In OptFuzz, we implement code abstraction based on GIMPLE, which we use as as an implementation choice. It is important to note that code abstraction could also be achieved using LLVM IR, allowing this approach to be extended to other programming languages that provide intermediate representations. Recognizing that program skeletons play a crucial role in triggering compiler bugs [19, 70, 90], OptFuzz performs code abstraction on GIMPLE statements to extract control-flow skeletons. Specifically, OptFuzz traverses GIMPLE statements within each basic block of the source code and identifies the type of each statement.

OptFuzz retains only control-flow–relevant GIMPLE statements—`GIMPLE_CALL`, `GIMPLE_COND`, `GIMPLE_GOTO`, `GIMPLE_LABEL`, `GIMPLE_RETURN`, and `GIMPLE_SWITCH`. It also retains all function call implementations contained in

[3]For the complete GIMPLE instruction set and all GIMPLE code types, refer to the official GCC documentation: https://gcc.gnu.org/onlinedocs/gccint/GIMPLE-instruction-set.html

Just Accepted

individual basic blocks to ensure semantic completeness of the control-flow structure. Non-control-flow statements such as GIMPLE_ASSIGN, GIMPLE_NOP, and GIMPLE_OMP_* are removed. Through this strategy, OptFuzz effectively extracts program control-flow skeletons while preserving the original control-flow structure and achieving significant code simplification.

Figure 8 illustrates an example of this process. OptFuzz use GIMPLE to traverse all basic blocks. For each statement, OptFuzz determine whether to remove it based on its statement type.

```c
int foo(int *a,int b){
  int c = 3, d = 0;
  for(int i = 0;i < b; i++){
    d += a[i];
    if(c == d)
      break;
  }
  return 0;
}

int main(){
  int b[3] = {1,1,1};
  int *p = b;
  foo(p,3);
  return 0;
}
```

(a) Before extraction.

```
foo (pointer_type, integer_type)
  if (c == d)
  if (i < b)
  return D.2849;

main (void)
  foo (p, 3);
  return D.2851;
```

(b) After extraction.

Fig. 8. Figure 8a is a simple source program. Following the rules proposed in Section 3.3, which involve extracting partial jumps, function definitions, and function calls, the corresponding IR segments are selected at the IR level for extraction, resulting in the outcome depicted in Figure 8b.

Just Accepted

The resulting abstracted programs allow LLMs to focus on core logic structure. This significantly reduces the complexity of the programs.

3.4 Compilation Option Generation

Different compilation options can control the compiler's behavior and explore more compiler paths. OptFuzz can generates a large number of suitable compilation options for testing. By testing different combinations of compilation options, we can more comprehensively evaluate the stability and reliability of the compiler.

Based on the abstracted code snippets, we use LLMs to generate complex compilation options that may trigger compiler defects. LLMs can understand the semantics of IR [51] and select appropriate compilation option combinations, fully leveraging its prior knowledge to make the generated compilation options more effective and able to explore compiler behavior more comprehensively.

We input the complete abstracted source code into the LLM. Segmentation is performed only when the input exceeds the maximum context length supported by the LLM. And, we construct a prompt to generate compilation options. The prompt requires LLM to generate suitable compiler options to comprehensively test compilers.

Prompt for generating complex compilation options

I will provide a code snippet, and I kindly request your expertise in providing a compilation option used for testing *compiler*. Please make sure that the compilation option be sufficiently intricate to thoroughly evaluate the compiler's capabilities.

3.5 Report Gathering and Deduplication

After generating the compilation options, we test the targeted compilers using the extracted code snippets as input programs. During the testing process, we record the compilation results produced with various groups of compilation options and analyze the resulting errors. We primarily focus on two types of defects: ICE (Internal Compiler Error) and Hang (compiler freeze) issues.

For ICE issues, we analyze the compiler's error messages and stack traces to determine the root cause of the error. When GCC crashes, it emits specific error codes such as **internal compiler error: in xxx, at XXX:xx**, including the function and file location. These highly specific identifiers allow us to distinguish between distinct crashes. LLVM provides similar assertion-based messages (e.g., `Assertion XXX failed`) with file and line information. We treat two crashes as duplicates only if both their messages and crash locations match. Therefore, we focus on both the error message and the location of the crash (function name, file, and line number) to achieve effective deduplication. For Hang issues, we record the compiler's execution time and resource usage to determine the problem. More specifically, by using GCC's `-ftime-report` option and LLVM's `-time-trace` option, you can analyze the execution time of each phase, such as parsing, optimization, and code generation, to identify potential bottlenecks causing performance issues.

OptFuzz employs this deduplication process by comparing the backtrace information and error details across both GCC and LLVM. The relevant fields, such as the error message, crash function, and file/line information, are isolated and compared against a database of historical bug reports that already recorded a specific defect in the compiler caused by a certain piece of code, as well as the entries documented within the GCC and the LLVM repository. This ensures that each identified bug is unique, improving the integrity and reliability of the bug deduplication process. As a result, only genuinely new defects are reported, minimizing redundant testing and enhancing the efficiency of the testing process.

In subsequent experiments, we will combine this bug-deduplication process with our experimental analysis. By comparing the effectiveness of different compilation option generation methods, we will further validate the superior performance of OptFuzz in bug detection, particularly in generating more complex options that enhance code coverage and uncover deeper, previously hidden compiler bugs.

This section details our method for compiler testing. By systematically documenting and scrutinizing compilation outcomes, OptFuzz is equipped to proficiently detect and address compiler anomalies, thereby improving the compiler's overall quality. Through the integration of bug reports, code abstraction, and the generation of intricate compilation options, OptFuzz demonstrates its capability to precisely identify and report compiler defects.

4 Evaluation

This section presents a comprehensive evaluation of OptFuzz, evaluating its effectiveness in compiler bug detection and efficiency in exploring the compilation space. In this paper, we answer the following research questions:

- **RQ1: Bug-finding Capability:** How many compiler bugs have been found by OptFuzz? (Section 4.2)
- **RQ2: Effectiveness:** How does OptFuzz perform when comparing against existing techniques? (Section 4.3)
- **RQ3: Sensitivity Analysis:** How does the performance and defect detection capability compare when intermediate representation extraction is not performed? (Section 4.4)
- **RQ4: Compilation Space Exploration:** Can OptFuzz generate diverse and high-coverage compilation options for exploring the compiler space? (Section 4.5)

Just Accepted

Table 2. Number of Historical Bug Reports and Corresponding Code Snippets

Report Type	GCC		LLVM	
	# Reports	# Code Files	# Reports	# Code File
With Attachment	31320	28592	2324	5736
With URL	1718	1902	1053	3096
Without Attachment/URL	46719	69246	1421	2367
Total	79757	99740	4798	11199

4.1 Implementation and Evaluation Setup

Implementation. OptFuzz is comprising over 4,000 lines of code in C++, Python, and shell scripting. It encompasses all aspects including program collection, code abstraction, generation of compilation options, and execution of tests. Besides, considering multiple factors such as performance and cost, we have integrated Gemini [71], for implementing OptFuzz.

Target Compilers. To ensure relevance, OptFuzz was applied to continuously test the latest development versions of two prevalent compilers: GCC and LLVM. Specifically, we sourced daily builds of GCC from its repositories and utilized the nightly packages of LLVM [49].

Test Programs. We extracted test programs from all reports of ICE, Crash, and other evidently defects that could trigger a problem in the GCC and LLVM at the commencement of our experiments. Following the process outlined in Section 3, we obtained around 110,939 test programs from 84,555 bug reports, including C and C++ source codes, which is shown in Table 2. This process is ongoing, and has been continuously conducted throughout the testing phase.

Hardware. Our evaluation was conducted on a workstation equipped with an Intel 10-core 2.30 GHz CPU, 118 GiB of RAM, and the Ubuntu 20.04.3 LTS operating system.

4.2 RQ1: Bug-finding Capability

Table 3 summarizes the bugs OptFuzz discovered during our testing period. Overall, we reported 64 unique bugs in total: 43 for GCC and 21 for LLVM. The developers have confirmed 53 of them as real bugs. Such results demonstrate the powerful bug-finding capabilities of OptFuzz. Of all these bugs, 24 of them have been fixed, of which 18 fixed bugs are in GCC and 6 fixed bugs are in LLVM.

Note that the bug management guidelines of GCC and LLVM are not exactly similar [66, 88]. Specifically, a reported bug in GCC Bugzilla will be initially labeled as "UNCONFIRMED", and the status will turn to "NEW" if developers confirm it. However, in LLVM issue tracker, a reported bug will be labeled as "new issue" by default, and if the developers confirm it, they will remove the "new issue" label and add other labels. For example, if a bug is related to link time optimization, it will be labeled as "LTO". In addition, if the status of a bug is "Assigned" means the bug is confirmed and is assigned to a certain developer to deal with it. If a bug is fixed by developers in LLVM issue tracker, the corresponding issue report will be closed.

Influence of bugs. To further understand the influence of our reported bugs, we investigate the official release versions of compilers that are affected by the reported bugs. As illustrated in Figure 9, one GCC bug persisted since its introduction in GCC 4, demonstrating its long-standing nature. Similarly, LLVM contains several long-lived bugs originating from LLVM 3, including two performance-critical bugs that affect all detectable LLVM releases. We note that this figure is intended as an approximate view rather than an exact per-version reproducibility table. In practice, some bugs may be confirmed on certain historical versions and also observed on the trunk

Just Accepted

Table 3. Bugs Reported by OptFuzz.

No.	Compiler	BugID	Status	Type	No.	Compiler	BugID	Status	Type
1	LLVM	95366	Fixed	ICE	**33**	GCC	115599	Fixed	ICE
2	LLVM	95420	Confirmed	ICE	**34**	GCC	115620	Unconfirmed	ICE
3	LLVM	95495	Fixed	ICE	**35**	GCC	115623	Fixed	ICE
4	LLVM	95500	Confirmed	ICE	**36**	GCC	115786	Unconfirmed	ICE
5	LLVM	97780	Confirmed	ICE	**37**	GCC	115787	Confirmed	ICE
6	LLVM	99516	Confirmed	Hang	**38**	GCC	115919	Fixed	ICE
7	LLVM	99641	Confirmed	Hang	**39**	GCC	115930	Unconfirmed	ICE
8	LLVM	112069	Confirmed	ICE	**40**	GCC	115940	Confirmed	ICE
9	LLVM	112086	Confirmed	ICE	**41**	GCC	116002	Fixed	Hang
10	LLVM	113803	Confirmed	ICE	**42**	GCC	116042	Unconfirmed	ICE
11	LLVM	114758	Confirmed	ICE	**43**	GCC	116113	Fixed	ICE
12	LLVM	114794	Confirmed	ICE	**44**	GCC	116320	Fixed	ICE
13	LLVM	116781	Confirmed	ICE	**45**	GCC	116323	Fixed	ICE
14	LLVM	117975	Fixed	ICE	**46**	GCC	117065	Fixed	ICE
15	LLVM	118058	Confirmed	ICE	**47**	GCC	117083	Confirmed	ICE
16	LLVM	118061	Confirmed	ICE	**48**	GCC	117091	Fixed	Hang
17	LLVM	135621	Fixed	ICE	**49**	GCC	117445	Fixed	ICE
18	LLVM	136119	Fixed	ICE	**50**	GCC	117806	Fixed	ICE
19	LLVM	138850	Fixed	ICE	**51**	GCC	117826	Fixed	ICE
20	LLVM	143091	Confirmed	ICE	**52**	GCC	117837	Confirmed	ICE
21	LLVM	143266	Confirmed	ICE	**53**	GCC	117845	Fixed	ICE
22	GCC	115411	Confirmed	ICE	**54**	GCC	117856	Unconfirmed	ICE
23	GCC	115412	Confirmed	ICE	**55**	GCC	117898	Fixed	ICE
24	GCC	115426	Fixed	ICE	**56**	GCC	118290	Confirmed	ICE
25	GCC	115431	Unconfirmed	ICE	**57**	GCC	119842	Confirmed	ICE
26	GCC	115469	Fixed	ICE	**58**	GCC	120557	Confirmed	ICE
27	GCC	115489	Confirmed	ICE	**59**	GCC	120558	Confirmed	ICE
28	GCC	115501	Fixed	ICE	**60**	GCC	120563	Unconfirmed	ICE
29	GCC	115510	Unconfirmed	ICE	**61**	GCC	120564	Confirmed	ICE
30	GCC	115560	Confirmed	ICE	**62**	GCC	120575	Fixed	ICE
31	GCC	115572	Confirmed	ICE	**63**	GCC	120602	Unconfirmed	ICE
32	GCC	115588	Unconfirmed	ICE	**64**	GCC	120618	Unconfirmed	ICE

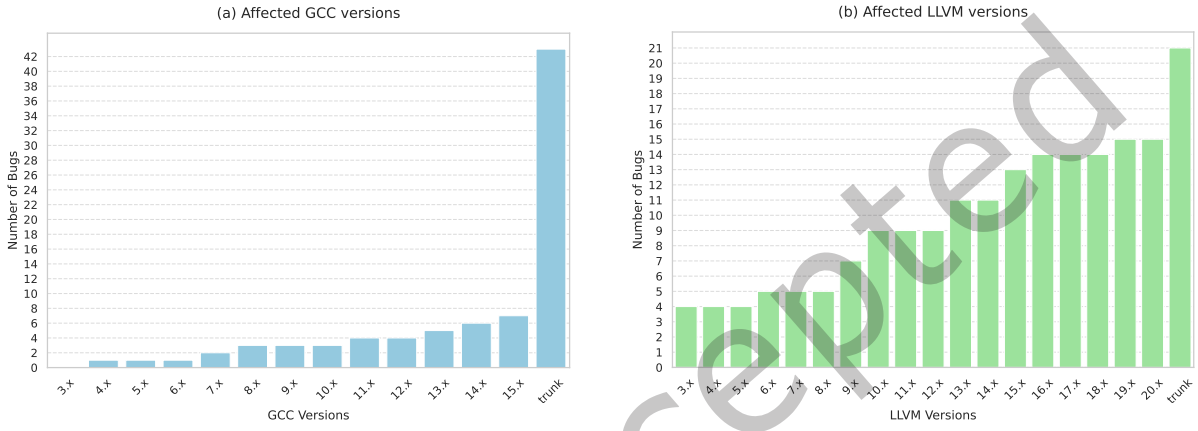
Just Accepted

version available at the time of validation, while not necessarily being reproducible on every intermediate version. Therefore, to better illustrate the overall distribution of affected versions, we adopt an approximate cumulative presentation. From this perspective, the figure suggests that many reported bugs are not limited to old releases and can still be observed in recent development versions. These observations highlight the capability of OptFuzz to uncover long-standing and widely propagated bugs, underscoring the practical significance of our bug-finding results.

Affected Compiler Components. As shown in Table 4, the 64 bugs identified by OptFuzz span various stages of the compiler pipeline, with the front-end being the most impacted (46 out of 64, or 71.9%). The middle-end accounts for 11 bugs (17.2%), indicating that our LLM-generated compilation options can effectively perturb internal compiler representations, exposing latent issues in intermediate transformation and analysis stages. Notably, 4 bugs are found in the optimization phase, showing that the generated options occasionally activate aggressive or corner-case optimizations. Finally, we observe 3 bugs in the back-end, reflecting OptFuzz's ability

Table 4. The number of affected compiler components.

Component	GCC	LLVM	Total
Front-End	34	12	46
Middle-End	5	6	11
Optimization	3	1	4
Back-End	1	2	3



(a) Affected GCC versions (b) Affected LLVM versions

Fig. 9. Approximate distribution of compiler versions affected by the bugs reported by OptFuzz. The figure is intended to illustrate the overall version span of the reported bugs rather than exact per-version reproducibility counts. In practice, some bugs may be observed on certain historical versions and also on the trunk version available at the time of validation, while not necessarily being reproducible on every intermediate version. Therefore, we adopt an approximate cumulative presentation here.

Just Accepted

to test across the entire compilation pipeline. These results demonstrate that our approach—combining historical bug-triggering programs and LLM-generated options—can thoroughly explore diverse compiler components, rather than being confined to shallow or surface-level bugs.

Feedback and discussions. After the bug reports are submitted, many of them invoke developers' lively discussion and may inspire their work in progress. 6 bugs were confirmed within 30 minutes after we reported them. The developers usually point out the root causes of the reported issues in the comments. After the bugs were fixed, some developers expressed their gratitude for our contributions to the GCC community. With respect to GCC bug 115623, the developer expressed his opinion that "*Fixed in master and GCC-14, thanks for the report!*" Also, for GCC bug 117898, the developer expressed his opinion that "*Mine, thanks for the bug report!*" According to the developers' response, we deduce that the bugs identified by OptFuzz are meaningful and contribute well to the compiler community.

4.3 RQ2: Effectiveness

Baseline approaches. To evaluate the bug finding effectiveness of OptFuzz, we compared OptFuzz with the following baseline techniques:

- **CSmith** and **YarpGen v2:** We use CSmith and YarpGen to generate test programs and use different optimization options[4] to test the compiler to verify the effectiveness of using programs in bug reports as the base test programs. CSmith is extensively used in the literature of C compiler testing [7, 37, 67] and effective in finding bugs. YarpGen is a random program generator, which produces correct runnable C/C++ programs. The generator is specifically designed to trigger compiler optimization bugs and is intended for compiler testing.
- **Random Compilation Options (Opt_{ran}):** The Opt_{ran} approach represents the conventional approach, where programs in the bug reports are paired with randomly selected compilation options. This method aims to explore the compilation space without any guidance mechanism, thus serving as a fundamental baseline for evaluating the effectiveness of OptFuzz.
- **Program Mutation ($P_{mutation}$):** To understand the effectiveness of using compilation options for compiler testing, we mutate the historical bug-triggering test program using MULL [15], which is a relatively mature and well-maintained C and C++ language mutation tool and do not test with compile options, which can evaluate the effectiveness of using compile options for compiler testing. During mutation testing, we applied all mutation operators from MULL to perform single mutation operations on each program. This process yielded a total of 112,282 valid mutants for C programs and 98,985 for C++ programs.
- **Opt_{ran} + $P_{mutation}$:** Opt_{ran} and $P_{mutation}$ represent two distinct, lightweight testing strategies. Given their computational efficiency and complementary approaches—Opt_{ran} exploring diverse compilation combinations while $P_{mutation}$ generating varied program inputs—we investigated their combination as a synergistic baseline. This integrated approach aims to leverage the strengths of both strategies for enhanced bug-finding capability and compiler code coverage while maintaining low resource overhead.
- **COTest:** COTest [7] utilizes machine-learning (the XGBoost algorithm) to model the relationship between test programs and compiler optimization settings, predicting the bug-triggering probability of a test program under an optimization setting. It also implements a diversity augmentation strategy to identify a subset of optimization settings for effective compiler testing. Similar to OptFuzz, COTest also construct compilation options for different programs for compiler testing. By comparing with COTest, we can evaluate the effectiveness of using LLMs for generating compilation options.
- **CTOS:** CTOS [34] utilizes a vector-based approach for LLVM's compilation options generation by representing optimization sequences and test programs via Doc2Vec embeddings. It employs a centroid selection scheme to identify diverse optimization sequences and programs, followed by differential testing to detect optimization-triggered bugs. While CTOS shares OptFuzz's goal of generating program-specific compilation options sequences, its core methodology fundamentally differs: CTOS relies on static vector representations and clustering algorithms to model optimization-program interactions, whereas OptFuzz leverages the generative capabilities of LLMs to dynamically synthesize compilation options. This comparison allows us to evaluate the effectiveness of using LLMs for generating compilation options. To measure the impact of historical bug-triggering programs on CTOS, we will also use historical defect programs combined with CTOS for subsequent testing.
- **Extracting code snippets using regex-based tool (Opt_{Regex}):** The Opt_{Regex} approach represents the conventional approach, where programs in the bug reports are extracted with regex-based tool. We implement Opt_{Regex} using manually designed regular expressions that target core C/C++ syntactic patterns (e.g., function definitions, preprocessor directives, and variable declarations). This approach explores the effectiveness of using simple and syntax-guided method to extract programs, thus serving as a fundamental baseline for evaluating the effectiveness of OptFuzz for using LLMs to extract code.

[4]For CSmith and YarpGen, we first use the programs directly generated by the respective generators. Following the practice of prior work [41], we test these programs under a set of commonly used optimization options, including `-O`, `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, `-Ofast` (i.e., `-O3 -ffast-math` for LLVM), `-Og`, and `-Oz`.

Just Accepted

- **Test compilers with real-world functions via Creal (Creal):** Creal [41] is a novel compiler-testing approach that injects real-world code snippets (e.g., functions from open-source projects like Linux, OpenSSL) into seed programs (e.g., generated by Csmith/YARPGen). It combines the diversity of real-world code with the structured validity of synthetic programs, uncovering deep compiler bugs missed by traditional random/mutation-based tools. By comparing with Creal, we can evaluate the effectiveness of using LLMs for extracting programs from bug reports.
- **Fuzz4All**: Fuzz4All [82] is a universal fuzzing tool that uses LLM to generate diverse and valid test inputs for various programming languages and systems. It introduces two key techniques: autoprompting, which distills user inputs into effective prompts, and an LLM-powered fuzzing loop, which iteratively updates prompts to generate new inputs. By comparing with Fuzz4All, we can evaluate the the effectiveness of directly using LLMs to generate program for compiler testing.
- **Generate test programs with LLM and test compilers with compilation options generated by LLM (LLMProgGen)**: LLMProgGen use an LLM to randomly generate test programs and ask it, using a prompt presented as same as OptFuzz in Section 3.4, to generate compilation options to find compiler bugs.

Table 5. Code Extraction Evaluation Criteria.

#	Metrics	Scoring Criteria
1	**Relevance (0–4)** Whether the extracted program is in the bug report (excluding patch, command lines, backtrace, etc.).	0: Not a program. 1: Irrelevant code, missing bug-triggering program. 2: Partially related code, but mixed with non-code discussion. 3: Related code was extracted, but mixed with stacktraces, patches etc. 4: Only relevant code was extracted from the bug report.
2	**Completeness (0–4)** Whether the extracted program is structurally complete (e.g., functions or structs not truncated).	0: Completely missing. 1: Partially extracted, but key components (e.g., entire functions) missing. 2: Mostly extracted, but some line-level content was missed. 3: Mostly complete with only minor omissions. 4: Complete and faithful to the original bug-report content.
3	**Multi-snippet Handling (0–2)** Whether the extraction handles multiple code blocks.	0: None of the relevant code snippets were extracted. 1: Some of the relevant code snippets were extracted. 2: All relevant code snippets were extracted completely.

Measurements. We conduct this evaluation by leveraging results from three distinct analytical perspectives. First, following the existing work [7–9], we first tested each approach for 24 hours and we used the number of detected bugs and crashes within the same testing period to measure the performance of OptFuzz. At the same time, we also tested each approach for 100,000 times compilations and we also used the number of detected bugs and crashes within the same testing compilation to measure the performance of OptFuzz. Second, to evaluate the effectiveness of OptFuzz in enhancing code coverage for programs, we collect the coverage information generated in 24 hours. An improvement in code coverage would suggest that our transformations effectively explore additional code paths within the compilers, thereby increasing the likelihood of triggering compiler bugs. Third, to comprehensively evaluate whether OptFuzz can accurately and completely extract C/C++ code snippets from real-world compiler bug reports, we design a systematic human-in-the-loop evaluation. We randomly sample 5% of bug reports from our full dataset and manually evaluate the code snippets extracted by OptFuzz and Opt_{Regex}. And to assess code extraction quality, we design a fine-grained scoring rubric consisting of three dimensions: **Relevance** (0–4), **Completeness** (0–4), and **Multi-snippet Handling** (0–2). The definitions of each metric are summarized in Table 5. As prior studies have not quantitatively evaluated the quality of extracted code in this manner, we introduce these metrics to enable a more rigorous and systematic comparison.

Number of detected bugs. Table 6 and 7 present the number of detected bugs and crashes within 24 hours and with 25,000 compilations for different techniques for GCC and LLVM. Csmith did not detect any new bugs during

Just Accepted

the evaluation period, likely because it has already been used extensively to test the historical compiler versions. Yarpgen found two bugs, both due to insufficient memory allocation. Compared with Opt_{ran}, which generated 3,949,528 options for GCC and 4,134,719 for LLVM in 24 hours, OptFuzz still outperformed it in bug detection. This evaluation further highlights the importance of rigorous crash deduplication: for instance, `-module-file-info` and `-objcmt-*` triggered over 1,000 redundant crashes with identical LLVM assertion failures. Table 7 also shows that Opt_{ran} requires a disproportionately large number of tests to discover bugs, suggesting that naïve random option generation is not an effective exploration strategy. Although $P_{mutation}$ produced 112,282 valid C mutants and 98,985 valid C++ mutants, semantic-preserving mutations provided limited improvement in bug-finding capability. COTest and CTOS detected no new bugs with Csmith-generated programs; CTOS rediscovered three known bugs when using historical triggering programs, but they overlapped with those found by Opt_{ran}, further highlighting OptFuzz's advantage. Further analysis shows that the bugs discovered by CTOS overlap to some extent with those found by Opt_{ran}, suggesting that its optimization-sequence generation strategy and training process may incorporate a degree of randomness.

These results suggest that CTOS's option-generation strategy and training methodology may introduce randomness, limiting its exploration effectiveness. They also highlight the value of historical bug-triggering programs for uncovering deeper compiler behaviors. Compared with Opt_{Regex}, our method demonstrates superior extraction and generation performance through LLM-based processing of bug reports. Although combining Opt_{ran} and $P_{mutation}$ yields some improvement, its limited performance under fixed-time constraints reveals inherent challenges in exploration efficiency. The compounded effects of both baselines tend to (1) reduce mutation targeting precision and (2) distribute testing resources suboptimally. While Fuzz4All demonstrates the effectiveness of its auto-prompting strategy, its reliance on the StarCoder LLM significantly reduces test case generation efficiency, thereby diminishing its overall effectiveness. While Fuzz4All benefits from its auto-prompting mechanism, StarCoder's latency significantly reduces test throughput. Finally, LLMProgGen—a naïve combination of LLM-generated test cases and compilation options—performs suboptimally, reinforcing that effective compiler fuzzing requires context-aware and semantically guided generation, as implemented in OptFuzz. These findings collectively validate the superior approach of OptFuzz.

We further analyzed the number of unique bugs that were only detected by OptFuzz, Opt_{ran}, $P_{mutation}$, Opt_{Regex}, Fuzz4all, Opt_{ran}+$P_{mutation}$, CTOS (Use historical bug-triggering programs) and YarpGen, with results shown in Figure 10. In particular, eight out of the bugs discovered by OptFuzz in this experiment are not detected by the

Table 6. Number of detected bugs and crashes within 24 hours for each method. Values in parentheses indicate the results obtained using historical bug-triggering programs as test inputs.

Method	GCC		LLVM		Total	
	Bug	Crash	Bug	Crash	Bug	Crash
CSmith	0	0	0	0	0	0
YarpGen	1	2	1	3	2	5
Creal	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
COTest	0	0	0	0	0	0
CTOS	–	–	0 (3)	0 (83)	0 (3)	0 (83)
LLMProgGen	0	0	0	0	0	0
Fuzz4all	1	2	4	13	5	15
OptFuzz	18	58	7	167	25	225
Opt_{ran}	13	31	4	1,364	17	1,395
$P_{mutation}$	14	29	3	60	17	89
Opt_{Regex}	7	54	2	36	9	29
Opt_{ran}+$P_{mutation}$	15	48	3	214	18	262

Just Accepted

Table 7. Number of detected bugs and crashes with 25,000 times compilations for each method.

Method	GCC		LLVM		Total	
	Bug	Crash	Bug	Crash	Bug	Crash
CSmith	0	0	0	0	0	0
YarpGen	0	0	0	0	0	0
Creal	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
COTest	0	0	0	0	0	0
CTOS	–	–	0 (1)	0 (33)	0 (1)	0 (33)
LLMProgGen	0	0	0	0	0	0
Fuzz4all	1	1	1	3	2	2
OptFuzz	5	14	5	18	10	32
Opt_{ran}	2	2	3	149	5	151
$P_{mutation}$	3	4	2	14	5	18
Opt_{Regex}	4	16	2	8	6	24
Opt_{ran}+$P_{mutation}$	1	1	3	3	4	4

baselines, demonstrating its unique value.

Code Coverage. Table 8 and Table 9 presents the code coverage achieved by OptFuzz and various baseline fuzzers for GCC and LLVM, respectively. The "Raw" columns show the total coverage achieved, while the "Improvement" columns indicate the increase in coverage compared to a baseline (implicitly, CSmith, which reports +0 improvement for its own coverage). For GCC (Table 8), OptFuzz achieved highest figures that significantly surpass all other methods evaluated. For instance, Opt_{ran} and $P_{mutation}$ individually show substantial improvements, which demonstrates both using compilation options and historical bug-triggering programs can significantly improve compiler coverage and greatly enhance the exploration of the compiler's internal behaviors. Their combination, Opt_{ran}+$P_{mutation}$, performed better than individual strategies, which demonstrates the two approaches explore orthogonal regions of the compiler space. But is still considerably outperformed by OptFuzz. Other LLM-based fuzzers, such as LLMProgGen and Fuzz4all, demonstrate that using LLMs to generate compilation options or test programs can lead to greater compiler coverage improvement. Dedicated

Table 8. Line, Function, and Branch coverage achieved for **GCC** at 24 hours. The **Improvement** is calculated based on the lowest-performing fuzzer (Csmith).

Target	Fuzzer	Line		Function		Branch	
		Raw	Improvement	Raw	Improvement	Raw	Improvement
GCC	**OptFuzz**	621,684	+504,140	59,612	+45,679	529,144	+441,799
	Opt_{ran}+$P_{mutation}$	529,042	+411,498	60,220	+46,287	432,805	+345,460
	Opt_{ran}	522,321	+404,777	59,356	+45,423	423,216	+335,871
	$P_{mutation}$	385,470	+267,926	37,671	+23,738	316,353	+229,008
	LLMProgGen	338,284	+220,740	38,645	+24,712	262,187	+174,842
	Fuzz4all	245,568	+128,024	27,877	+13,944	187,822	+100,477
	COTest	240,478	+122,934	25,427	+11,494	191,164	+103,819
	Opt_{Regex}	178,276	+60,732	20,518	+6,585	133,760	+46,415
	YarpGen	148,489	+30,945	17,421	+3,488	110,330	+22,985
	CReal	134,173	+16,629	15,377	+1,444	98,348	+11,003
	CSmith	117,544	+0	13,933	+0	87,345	+0

Just Accepted

Table 9. Line, Function, and Branch coverage achieved for **LLVM** at 24 hours. The **Improvement** is calculated based on the lowest-performing fuzzer (Csmith).

Target	Fuzzer	Line		Function		Branch	
		Raw	Improvement	Raw	Improvement	Raw	Improvement
LLVM	**OptFuzz**	1,137,284	+688,230	65,976	+46,404	413,965	+335,204
	Opt_{ran}	1,040,233	+591,179	61,275	+41,703	345,273	+266,512
	Opt_{ran}+P_{mutation}	992,836	+543,782	58,064	+38,492	358,430	+279,669
	LLMProgGen	779,977	+330,923	40,934	+21,362	222,685	+143,924
	$CTOS_{\text{His}}$	698,657	+249,603	37,843	+18,271	192,784	+114,023
	P_{mutation}	589,357	+140,303	29,221	+9,649	173,125	+94,364
	Fuzz4all	567,706	+118,652	28,238	+8,666	140,791	+62,030
	Opt_{Regex}	547,436	+98,382	26,831	+7,259	126,290	+47,529
	CTOS	460,806	+11,752	21,022	+1,450	85,728	+6,967
	CReal	460,449	+11,395	21,180	+1,608	86,269	+7,508
	COTest	458,035	+8,981	20,602	+1,030	83,347	+4,586
	YarpGen	452,105	+3,051	21,833	+2,261	84,288	+5,527
	CSmith	449,054	+0	19,572	+0	78,761	+0

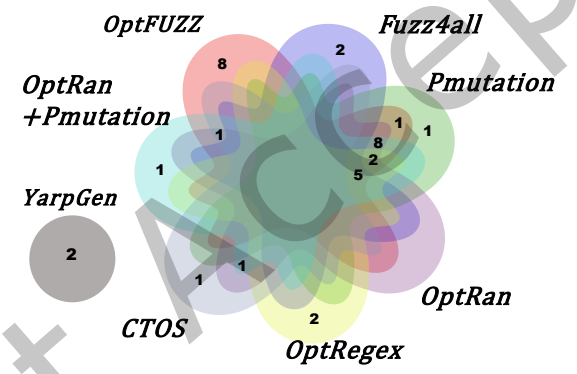


Fig. 10. OptFuzz, Opt_{ran}, $P_{mutation}$, Opt_{Regex}, Fuzz4all, Opt_{ran}+$P_{mutation}$, CTOS (Use historical bug-triggering programs) and YarpGen.

compiler testing tools like COTest and CTOS indicate that leveraging complex compilation options can enhance coverage. CTOS, when used with historical bug-triggering programs ($CTOS_{\text{His}}$), improves line, function, and branch coverage significantly compared to its original configuration. This further validates the importance of test program quality in compiler testing. CReal further highlights that extracting code from real-world projects can improve compiler coverage. Opt_{Regex} also achieves less substantial improvements compared to OptFuzz. The superior performance of OptFuzz is attributed to its strategy of leveraging LLMs to generate compilation options for test programs and the use of historical bug-triggering programs. This approach allows OptFuzz to produce more diverse and effective option sequences that trigger deeper and previously unexercised code paths within the compiler, leading to higher coverage compared to purely random generation or mutation-based methods. These results indicate that OptFuzz can substantially enhance the achieved code coverage of compilers during testing. By exercising additional code paths in the compilers, OptFuzz not only improves the thoroughness of

Just Accepted

Table 10. Detailed Score Summary of Opt_{Regex} vs. OptFuzz in code extraction.

Metric	Method	Min	Q1	Median	Q3	Max	Average
Relevance (0–4)	Opt_{Regex}	1.00	2.00	3.00	3.00	4.00	2.44
	OptFuzz	1.00	4.00	4.00	4.00	4.00	3.86
Completeness (0–4)	Opt_{Regex}	1.00	1.00	2.00	2.00	4.00	2.08
	OptFuzz	1.00	4.00	4.00	4.00	4.00	3.83
Multi-snippet (0–2)	Opt_{Regex}	0.00	1.00	1.00	1.00	2.00	1.11
	OptFuzz	0.00	2.00	2.00	2.00	2.00	1.80
Total (0–10)	Opt_{Regex}	2.00	4.00	6.00	7.00	10.00	5.63
	OptFuzz	2.00	10.00	10.00	10.00	10.00	9.49

compiler testing but also increases the potential for uncovering hidden bugs, thereby demonstrating the practical value of OptFuzz.

Effectiveness of Code Extraction from Bug Reports As shown in Table 10, OptFuzz significantly outperforms Opt_{Regex} in all three evaluation metrics. It achieves a high average relevance and completeness score, showing its ability to capture real and well-structured code snippets. Moreover, its multi-snippet score indicates that it can reliably handle multiple code fragments within a single bug report. The average total score of OptFuzz reaches 9.49, compared to only 5.63 for Opt_{Regex}. To further evaluate OptFuzz's extraction capability, we additionally compare it with CReal, a recent framework designed for extracting executable functions from real-world open-source projects for fuzzing. While CReal primarily focuses on reusable code from large codebases, OptFuzz is tailored for extracting minmal yet bug-triggering snippets from noisy, unstractured bug reports. On the same dataset, CReal was able to extract only 3,869 function-level code snippets, whereas OptFuzz successfully identified a substantially larger number and more diverse set of code fragments suitable for fuzzing, as summarized in Table 2 (Section 4.1). In total, OptFuzz successfully extracted around 100,000 code fragments from GCC bug reports. Additionally, CReal is limited to extracting functions from source code files that are both compilable and contain a main function, thus rendering it less effective for extracting code snippets from bug reports. This highlights the domain-specific advantage of OptFuzz in compiler bug analysis and reproduction. These results confirm the superior capability of OptFuzz in identifying, extracting, and structuring C/C++ code snippets from unstructured bug report text. Compared to both handcrafted regex rules and existing code mining tools, OptFuzz offers a targeted, high-fidelity solution for downstream compiler testing tasks.

Case Study: Side-by-Side Extraction Example. To provide qualitative insight, we present a representative case study from GCC bug report #78648 in Figure 11. This report contains compiler diagnostics, stack traces, and the actual bug-triggering program in the same report text, which makes extraction difficult for approaches that rely on surface patterns. As shown in the figure, Opt_{Regex} extracts multiple fragments mixed with compiler diagnostics and incomplete code lines, whereas OptFuzz correctly identifies the complete bug-triggering program without including unrelated diagnostic information. The scores in this example can be interpreted according to the criteria in Table 5. For relevance, Opt_{Regex} receives a score of 3 because it does extract related code, but the extracted result is mixed with non-program content such as compiler diagnostics, which matches the definition of score 3 in our rubric. In contrast, OptFuzz receives a score of 4 because it extracts only the relevant bug-triggering program from the bug report. For completeness, Opt_{Regex} receives a lower score because its result consists of fragmented and partially truncated code rather than a complete and faithful reconstruction of the reported program, whereas OptFuzz receives a score of 4 because it preserves the full program structure as it appears in the report. For

Just Accepted

```
$ gcc-trunk small.C
small.C: In function 'void bar()':
small.C:4:14: error: invalid use of cv-qualified type
'const void' in parameter declaration
   e([](const void) {});
              ^~~~

small.C: In instantiation of
'void e(F) [with F = bar() [with int <anonymous> = 1]::<lambda>]':
small.C:1:28: internal compiler error: Segmentation fault
 template <typename F> void e(F) {}
                            ^
0xdf7cff crash_signal
        ../../gcc-source-trunk/gcc/toplev.c:333
...
...

$ cat small.C
template <typename F> void e(F) {}

template <int> void bar() {
  e([](const void) {});
}

void baz() { bar<1>; }
$
```

(a) Raw content from GCC bug report #78648. The bug report contains compiler diagnostics, stack traces, and the actual bug-triggering program in the same text.

```
void bar():
error: invalid use of
cv-qualified type 'const void'
e([](const void) {}

template <typename F> void e(F)
template <int> void bar()
[](const void) {}
void baz() { bar<1>; }
```

(b) Opt_{Regex} extraction result.

```
template <typename F> void e(F) {}

template <int> void bar() {
  e([](const void) {});
}

void baz() { bar<1>; }
```

(c) OptFuzz extraction result.

Metric	Opt_{Regex}	OptFuzz
Relevance	3	4
Completeness	2	4
Multi-snippet Handling	1	2
Total	6	10

(d) Evaluation scores.

Fig. 11. A qualitative case study of program extraction on GCC bug report #78648. The left panel shows the raw issue report, while the right panels show the extraction results of Opt_{Regex} and OptFuzz, together with their corresponding scores. Opt_{Regex} extracts incomplete fragments polluted by compiler diagnostics, whereas OptFuzz accurately recovers the complete bug-triggering program.

multi-snippet handling, Opt_{Regex} receives a score of 1 because it captures only part of the relevant program content in a fragmented manner, while OptFuzz receives a score of 2 because it successfully extracts the complete relevant code block. We also note that some existing tools, such as CReal, can only extract programs from certain well-structured and regular input formats. Therefore, when applied directly to raw bug reports such as the one shown in Figure 11, they may fail to produce any extraction result.

4.4 RQ3: Sensitivity Analysis

Baseline approach. To evaluate the efficacy of our devised IR-based code representation strategy, we crafted the following experiments and analyzed its contribution. Particularly, we designed a variant of OptFuzz$_{w/o\ Abs}$.

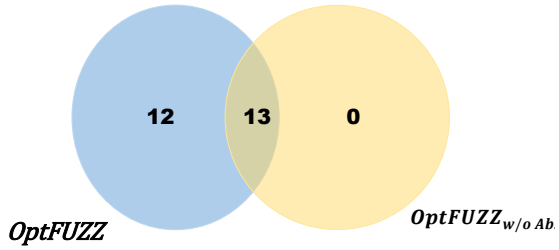
- OptFuzz$_{w/o\ Abs}$ (OptFuzz without code Abstraction): To isolate and understand the contribution of OptFuzz, we evaluate OptFuzz without incorporating code extraction at the IR-level. OptFuzz$_{w/o\ Abs}$ helps in quantifying whether the abstraction of the IR-level code influences the quality of the generated compilation options.

To ensure a fair comparison, OptFuzz and OptFuzz$_{w/o\ Abs}$ use the same LLM-extracted seed programs as described in Sec 4.1, the same testing budget (24 hours and 100,000 compilations), and the same bug-deduplication procedure described in Sec 3.5.

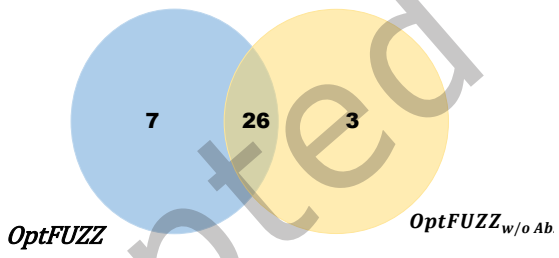
Just Accepted

Table 11. The left part means the proportion of successful compilations for OptFuzz and OptFuzz$_{w/o\ Abs}$ generated and tested within 24 hours. The right part means number of compilations of OptFuzz and OptFuzz$_{w/o\ Abs}$ generated and tested within 24 hours.

	Compilation Success Proportion (%)		Compilations Tested (24h)	
Compiler	OptFuzz	OptFuzz$_{w/o\ Abs}$	OptFuzz	OptFuzz$_{w/o\ Abs}$
GCC	79.42	73.47	20,314	12,063
LLVM	83.75	81.27	37,028	17,523
Total	82.21	78.09	57,342	29,586



(a) Relations of Unique Bugs Found by OptFuzz and OptFuzz$_{w/o\ Abs}$ within 24h.



(b) Relations of Unique Bugs Found by OptFuzz and OptFuzz$_{w/o\ Abs}$ within 100,000 times of compilation.

Fig. 12. Relations of Unique Bugs Found by OptFuzz and OptFuzz$_{w/o\ Abs}$.

The proportion of successful compilations The left part of Table 11 shows the proportion of successful compilations over 24 hours. When the compiler options input cause the compilation to fail with a error message of arguments, we believe that this is a failure caused by the compiler options. Here we define the compilation success rate as the proportion of all compiled files that are not compiled failed caused by compilation options. We also compare the proportion of successful compilations of OptFuzz, OptFuzz$_{w/o\ Abs}$, and Opt_{ran} running for 24 hours. We can find from the table that around 82% of the compilation options generated by OptFuzz can be successfully compiled demonstrating its effectiveness. For OptFuzz$_{w/o\ Abs}$, the number is similar to OptFuzz, around 80%, but slightly lower. We deduce that this difference is due to the presence of long code segments processed by OptFuzz$_{w/o\ Abs}$, exceeding the context length of LLMs and thus hindering their understanding of the complete test programs.

Number of unique bugs. We also analyzed the number of unique bugs detected by OptFuzz and OptFuzz$_{w/o\ Abs}$, with results shown in Figure 12. As illustrated in this figure, OptFuzz detects 25 unique bugs in total, 12 of which cannot be detected by OptFuzz$_{w/o\ Abs}$. This indicates the IR-based code abstraction strategy contributes to OptFuzz's bug-finding ability. As shown in Figure 12, OptFuzz$_{w/o\ Abs}$ uniquely finds three bugs. This suggests that the two variants do not always behave in exactly the same way on every individual case. Overall, IR abstraction still improves the total effectiveness. of OptFuzz's bugs are unique, demonstrating its unique value.

Impact of Code Abstraction. We observe that OptFuzz$_{w/oAbs}$ completed fewer compilations than OptFuzz within the same time budget. This is mainly because the longer, unabstracted inputs slow down the LLM's response time, reducing throughput in compilation option generation. To further evaluate whether the performance difference is solely due to this efficiency gap, we conducted a follow-up experiment where both approaches were constrained to perform the same number of compilations (100,000). Under this setting, the performance gap between the two methods narrowed, but OptFuzz still found more bugs. We also compared the

Just Accepted

Table 12. 24-hour Coverage Comparison between OptFuzz and OptFuzz$_{w/o\ Abs}$ on GCC and LLVM.

Method	GCC Coverage			LLVM Coverage		
	Line	Function	Branch	Lines	Functions	Branches
OptFuzz	**621,684**	**59,612**	**529,144**	**1,137,284**	**65,976**	**413,965**
OptFuzz$_{w/o\ Abs}$	601,697	58,794	508,211	1,109,125	64,517	396,313

Table 13. Time to Generate Compilation Options for Test Cases

Compiler	Techniques	Total	Average
GCC-15.0	OptFuzz$_{w/o\ Abs}$	163h43m10s	5.31s
	OptFuzz	118h19m43s	3.83s
LLVM-19.0	OptFuzz$_{w/o\ Abs}$	126h12m52s	4.10s
	OptFuzz	112h20m41s	3.64s

number of unique bugs discovered under the same compilation budget. Under this condition, the difference in bug-finding performance between the two approaches became much smaller. For OptFuzz, it found 19 unique bugs in GCC, 14 unique bugs in LLVM. And for OptFuzz$_{w/o\ Abs}$, it found 15 unique bugs in GCC, 14 unique bugs in LLVM. It suggest that the abstraction's primary benefit lies in improving efficiency rather than directly altering the effectiveness of LLM-generated options. These results validate the importance of abstraction in enhancing both the scalability and effectiveness of LLM-based compiler testing.

Code coverage. Table 12 shows the statistics of code coverage achieved by OptFuzz and OptFuzz$_{w/o\ Abs}$. The results indicate that OptFuzz improves the achieved code coverage of compilers slightly. Compared with OptFuzz$_{w/o\ Abs}$, OptFuzz enhances the thoroughness of compiler testing by a higher coverage, which demonstrates the practical value and efficacy of OptFuzz.

Time of compilation options generation. We conducted a comparative analysis to ascertain the time demands placed on OptFuzz and OptFuzz$_{w/o\ Abs}$ in generating compilation options for each individual programs, as illustrated in Table 13. Additionally, we also performed a comparative analysis to evaluate the frequency of compilation executions conducted by OptFuzz and OptFuzz$_{w/o\ Abs}$ within an equivalent temporal framework about 24 hours, as illustrated in the right part of Table 11.

As illustrated in Table 13, it shows that the time required to individually generate compilation options for all 110,939 source files is remarkably substantial. In general, the time needed to generate compilation options for GCC is relatively longer than that for LLVM, likely due to the more complex compilation options inherent in GCC. Concurrently, the results indicate that, after employing the code extraction method, OptFuzz has accelerated the generation of GCC and LLVM compilation options by 28.33% and 9.66%, respectively, compared to OptFuzz$_{w/o\ Abs}$. The findings presented in the right part of Table 11 also corroborate this perspective. Within 24 hours, OptFuzz performed 57,342 rounds of compilation option generation and compiler testing, exceeding OptFuzz$_{w/o\ Abs}$ by 94%, specifically 27,756 more instances. This result differs slightly from the previous one, which only involved compilation option generation, due to the inclusion of compilation testing in the experiment. This outcome demonstrates that by extracting key information from lengthy code and inputting it into LLMs, the efficiency of generating compilation options can be effectively enhanced. This substantiates the high efficiency of OptFuzz.

Just Accepted

4.5 RQ4: Compilation Space Exploration

This research question evaluates whether the compilation options generated by OptFuzz exhibit sufficient diversity and contextual relevance to meaningfully explore the compiler testing space. We also compare the effectiveness of OptFuzz against Opt_{Ran}, **CTOS** and **COTest**.

Modern compilers such as GCC and LLVM expose thousands of configurable compilation options, including optimization levels (e.g., `-O1`, `-O2`, `-Ofast`), target-specific settings (e.g., `-march=native`), code generation toggles (e.g., `-fomit-frame-pointer`), and warning controls (e.g., `-Wno-unused`). A successful compiler testing framework must not only generate combinations that expose bugs, but also ensure broad and diverse coverage over this massive compilation space. If the generated options are repetitive, shallow, or only explore a small fraction of the whole compilation space, the testing may miss critical behaviors hidden behind less common configurations. This research question evaluates whether OptFuzz is capable of producing a *diverse*, *non-redundant*, and *broadly covering* set of compilation options.

Measurements. We employ two metrics to answer this research question: (1) **Uniqueness of Options,** defined as the number of distinct compilation flags (e.g., `-O2`, `-fwrapv`) produced across all generated combinations; and (2) **Structural Diversity,** which characterizes the variation among option sets in terms of their composition and structure. To assess structural diversity, we compute pairwise Jaccard and cosine similarity scores across all generated option sets.

Table 14. Number of unique compilation options and compilation success rate (%) of GCC and LLVM for each method.

Method	Unique Options		Success Rate (%)	
	GCC	LLVM	GCC	LLVM
OptFuzz	**753**	**577**	**79.42%**	**83.75%**
Opt_{Ran}	2,406	1,990	20.22%	31.35%
CTOS	—-	71	—-	92.67%
COTest	249	77	94.35%	89.67%

How many unique options does OptFuzz generate? To quantify uniqueness, we collected all official options from the GCC-15 and LLVM-19 documentation, resulting in 2,474 unique GCC options and 2,098 LLVM options. We then analyzed the options produced by OptFuzz, Opt_{Ran}, **CTOS**, and **COTest** during the 24-hour experiment in RQ2 (Section 4.3). Note that, **CTOS** is designed specifically for LLVM, so its GCC results are omitted. As shown in Table 14, OptFuzz generates substantially more distinct and valid options than **CTOS** and **COTest**, whose outputs are constrained by predefined optimization templates. While Opt_{Ran} covers more options overall, it exhibits very low success rates (20.22% for GCC and 31.35% for LLVM). In contrast, OptFuzz achieves success rates of 79.42% for GCC and 83.75% for LLVM, significantly outperforming Opt_{Ran} (20.22% and 31.35%, respectively). When considering only valid options, the effective coverage of OptFuzz and Opt_{Ran} becomes comparable. Although **CTOS** and **COTest** achieve high success rates (above 89%), their limited scope prevents broad exploration of the configuration space. These results show that OptFuzz achieves a desirable trade-off between *diversity* and *validity*, producing a large and semantically meaningful, and compilable set of options without relying on handcrafted templates or domain-specific heuristics.

How structurally diverse are the generated combinations? To quantify the diversity of compilation options generated by OptFuzz, we compare the option sets produced by OptFuzz, Opt_{Ran}, **CTOS** and **COTest**. Followinw the existing work [31, 39, 62], we use three complementary metrics: the Jaccard similarity coefficient for set comparison and cosine similarity for vector-space analysis. The formal definitions and computational methods are detailed in Table 15.

Just Accepted

Table 15. Definitions of diversity metrics used to quantify structural variation between option sets.

Metric	Formula	Interpretation
Jaccard Similarity	$J(A,B) = \frac{\lvert A \cap B \rvert}{\lvert A \cup B \rvert}$	Element overlap between two sets; lower values indicate higher diversity.
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\lVert A \rVert \cdot \lVert B \rVert}$	Measurement of similarity between two non-zero vectors defined in an inner product space.

Table 16. Uniqueness ratio and similarity metrics for different methods on GCC and LLVM.

Metric	OptFuzz		Opt_{Ran}		CTOS		COTest	
	GCC	LLVM	GCC	LLVM	GCC	LLVM	GCC	LLVM
Jaccard Similarity								
Mean	0.0469	0.0563	0.0053	0.0039	–	0.0930	0.7713	0.5525
Median	0.0000	0.0000	0.0000	0.0000	–	0.0870	0.7713	0.5522
Min	0.0000	0.0000	0.0000	0.0000	–	0.0000	0.6872	0.3333
Max	1.0000	1.0000	0.1354	0.1429	–	1.0000	0.8626	0.7742
Std. Dev.	0.1191	0.1272	0.0135	0.0116	–	0.0723	0.0207	0.0533
Cosine Similarity								
Mean	0.1274	0.1379	0.1161	0.1750	–	0.2853	0.9972	0.8330
Median	0.0000	0.0000	0.1042	0.1675	–	0.2831	0.9973	0.8359
Min	0.0000	0.0000	0.0000	0.0000	–	0.0000	0.9939	0.5999
Max	1.0000	1.0000	0.6081	0.5981	–	1.0000	0.9986	0.9399
Std. Dev.	0.2120	0.2141	0.0851	0.0955	–	0.1428	0.0005	0.0365

Just Accepted

Table 16 presents a comprehensive comparison of the similarity statistics for the generated compiler option sets across four methods: OptFuzz, Opt_{Ran}, **CTOS**, and **COTest**, evaluated on both GCC and LLVM. The average Jaccard similarity for OptFuzz is relatively low, suggesting that the generated combinations are structurally distinct. Moreover, OptFuzz exhibits a substantially larger standard deviation, which indicates that while some combinations are similar, many are highly diverse or even disjoint. This wide distribution of similarity scores implies that OptFuzz covers a wide spectrum of the option space. In terms of cosine similarity, OptFuzz again demonstrates moderate average similarity with a high standard deviation. This suggests that OptFuzz is not restricted to fixed patterns and is capable of generating highly varied option sequences. In contrast, **CTOS** and **COTest** display much narrower distributions. For example, **COTest** has extremely high mean similarity and near-zero standard deviation, indicating that their generated option sets are largely homogeneous. While this may ensure stability, it limits the ability to explore compiler behaviors. Overall, the results highlight the superior exploration ability of OptFuzz. By producing more unique and structurally diverse combinations, OptFuzz enables broader coverage of the compiler's option space, which is essential for effective fuzzing and bug discovery.

To provide qualitative insight into the option generation results, we further analyze a representative example based on GCC bug #120564 in Figure 13. The input program contains an OpenMP pragma together with a nested loop structure, which suggests that OpenMP-related and loop-related compiler behaviors are relevant to bug triggering. In this case, OptFuzz generates option sequences including `-fopenmp` together with optimization and warning-related flags, which are closely aligned with the structural characteristics of the program. By contrast,

```
extern int b[];
void foo (){
  unsigned long v1, v2, v3;
#pragma omp parallel for schedule(static, 32) collapse(3)
  for (v1 = 0; v1 < 20; v1 += 2)
    for (v2 = 92233720368547758O7LL; v2 > 92233720368547758O7LL; v2 -= 3)
      for (v3 = 10; v3 > 0; v3--)
        b[64]++;}
```

(a) Input program from GCC bug #120564.

```
1. -O2 -fopenmp -Wno-omp-pthreads
2. -O3 -fopenmp -funroll-loops
```

(b) Option sequence generated by OptFuzz.

```
1.-fno-printf-return-value -fsave-optimization-record -funroll-loops -fsched-last-insn-heuristic -fmove-loop-stores -fno-
    guess-branch-probability -fipa-vrp -fmodulo-sched
2.-fivopts -fmin-function-alignment=10 -fipa-sra -freorder-blocks -fmove-loop-stores -flive-range-shrinkage -fmin-function-
    alignment=20 -fprofile-values -fprofile-use=path -fstore-merging -fprefetch-loop-arrays -ffinite-loops -fbranch-
    probabilities -ftrivial-auto-var-init -fearly-inlining
3.-ftree-slsr -frounding-math -fmerge-constants -fsched-stalled-insns-dep=5 -ftree-loop-im -fcprop-registers -funswitch-loops
     -fcprop-registers -fsched-rank-heuristic -fselective-scheduling2 -fipa-pta
```

(c) Option sequences generated by Opt_{Ran}.

```
hotcoldsplit -> extract-blocks -> forceattrs -> sink -> mergereturn -> float2int -> lcssa -> float2int -> alignment-from-
    assumptions -> loop-load-elim -> lcssa -> loop-data-prefetch -> loop-sink -> mergereturn -> loop-simplifycfg
```

(d) A representative optimization-pass sequence generated by CTOS.

Fig. 13. A qualitative comparison of option generation on GCC bug #120564. The input program contains an OpenMP pragma and nested loops. OptFuzz generates an option sequence including `-fopenmp`, which directly reflects the key OpenMP-related feature in the program. By contrast, Opt_{Ran} produces longer but more generic GCC optimization combinations, while CTOS generates an optimization-pass sequence that does not explicitly capture the OpenMP-specific structure.

Opt_{Ran} and CTOS produce more generic optimization combinations that do not directly reflect the key OpenMP-related feature in the input. This example illustrates that the advantage of OptFuzz does not come from generating arbitrarily complex option combinations. Instead, the LLM-based generation in OptFuzz tends to produce options that are better matched to the input program structure, which helps explore compiler behaviors more relevant to the given code. We also note that COTest is not included in this example because it is only applicable to Csmith-generated programs and therefore cannot be directly applied to this bug-triggering input. For reference, COTest may generate very large option configurations, e.g., `-fno-aggressive-loop-optimizations -fno-align-functions -fno-align-jumps -fno-align-labels -fno-align-loops ... -fno-allocation-dce`.

5 Discussion

In this section, we further discuss the results of our experiments.

5.1 Generality

In this study, we implement OptFuzz with a primary focus on GCC and LLVM. However, OptFuzz can be generalized to support other compilers targeting various programming languages, such as WebAssembly compilers.

Just Accepted

Specifically, many open-source compiler projects maintain issue trackers that contain a wealth of test programs derived from reported issues. OptFuzz leverages LLMs to effectively extract relevant programs from these bug reports. Moreover, most compilers also contain numerous options, which are supposed to be analyzed by LLMs, enabling comprehensive and systematic exploration for testing purposes. OptFuzz demonstrates the potential for extension to other compilers, and we intend to investigate this direction in future work.

5.2 Impact of Historical Bug Report on Bug Finding

To understand how the accumulation of historical bug reports impacts the bug-finding capability of OptFuzz, we analyze the source of each bug discovered in the 24-hour experiments presented in RQ2. Specifically, we examine which historical bug reports were responsible for triggering the final test programs that exposed compiler bugs. Interestingly, we found that **44%**(11/25) of these bugs were triggered by programs from bug reports submitted more than a decade ago (before 2015). This observation demonstrates that even very old bug reports can still uncover previously unknown or reoccurring compiler issues when paired with diverse and context-sensitive compilation options. It also confirms the cross-compiler applicability of such programs: many programs extracted from historical GCC bug reports were able to crash LLVM, and vice versa. These results highlight that even older bug reports—some submitted over a decade ago—can still provide valuable and reusable test programs. When paired with our LLM-based compilation option generation strategy, these "legacy" reports continue to reveal new issues in modern compilers. This finding supports the idea that historical bug repositories serve as a long-lasting, rich resource for compiler testing, and that OptFuzz can effectively harness this potential.

5.3 Threats to Validity

The use of IR. Step ② (Section 3.3), involves using IR for abstracting the source code. We have chosen GIMPLE here, GCC's IR over LLVM IR for this step. GCC employs different IR at various compilation stages, such as GIMPLE and RTL (Register Transfer Language). GIMPLE is utilized for intermediate optimizations, whereas RTL is used for backend code generation [23, 25, 51]. We chose GIMPLE merely as an implementation of an IR-based code abstraction method, noting that similar results could be achieved using LLVM IR [11]. Moreover, this strategy can also be attempted to be extended to other programming languages that have IR.

The Selection of LLM. Step ① (Section 3.2) and Step ③ (Section 3.4) of OptFuzz involve operations combined with LLMs, which raises the issue of selecting a specific LLM. After thorough comparisons among various LLMs, including ChatGPT-4 [59], CodeLlama [64], Incoder [18], Gemini [71], and ChatGLM [17], Deepseeek [12] etc, we found that most models can meet the requirements of OptFuzz, which is capable of accurately extracting potential programs from bug reports and possesses sufficient prior knowledge to generate compilation options for the given programs. We also tested running LLMs locally, but unexpectedly, performance was below expectations. We speculate this might be due to the parameter sizes of open-source LLMs. Considering multiple factors such as performance and cost, we selected Gemini for our experiments with default configuration. We mainly use the Gemini Flash series in our experiments, including `gemini-1.5-flash` and `gemini-2.0-flash-001`. As some API models are periodically updated or deprecated by the provider, later experiments were conducted with updated Flash variants such as `gemini-2.0-flash-lite` and `gemini-2.5-flash`. Theoretically, OptFuzz can be integrated with any LLM. We also evaluated the potential hallucination issues of LLMs in extracting code snippets from bugreports. Our experiments indicated that when the bugreport being processed did not contain code snippets, depending on the prompts used, LLM might output some irrelevant code. However, when the bugreport definitely contained code snippets, in almost all cases, LLM would faithfully and completely extract the code snippets within it. This also provides a guarantee for utilizing LLM to extract code snippets from bugreports.

The impact of uncertainty. This investigation is susceptible to several primary threats that warrant consideration. First, the presence of randomness in the all processes could introduce variability into the results of experiments.

Just Accepted

Secondly, the selection of test programs and the specific compiler versions also significantly affect the evaluation of OptFuzz. To address this, we ensured that all test programs were sourced from bug reports been fixed prior to the compiler's build time.

6 Related Work

6.1 Compiler Testing Techniques

The identification of compiler bugs has been an extensive research subject. Widely-used tools such as Csmith [87], a program generator specifically designed for C, have successfully uncovered thousands of compiler crashes and correctness bugs. And to enhance the bug finding capabilitiy of Csmith, researchers have mined fuzzer configurations (e.g., grammar and statement probabilities) from historically bug-triggering programs for more diverse generation, achieving notable results [8, 9]. Another approach, Equivalent Modulo Input (EMI) [36], proposes mutating or transforming seed programs while preserving their semantics under the same input. EMI can be implemented through techniques like deleting dead statements [36], inserting new code in inactive regions [37], or synthesizing equivalent code in active regions [67]. YARPGen [45, 46] is another C/C++ program generator specifically focusing on testing scalar optimizations. In terms of optimization options, COTest [7] uses machine learning methods to predict optimization options that may cause compiler defects. Wang et al. [77] presented a novel method to enhance black-box compiler option fuzzing by leveraging command feedback, such as error messages, standard output, and compiled files, to guide error fixing and option pruning through LLMs. However, their method does not target the generation of compiler options for source code; it merely utilizes large models to address issues in option repair. In addition to optimization correctness, other compiler issues, including incorrect debug information [42, 78] and missed optimizations [72], have been extensively studied. For instance, Li et al. [42] propose "actionable programs" to validate debug information for optimized code. Dfusor [78] transforms a seed program into multiple variants to identify inconsistencies in debug information. Another tool, Dead [72], injects markers to identify missed dead code eliminations in optimizing compilers. Programs from historical bug reports contain rich program semantics, which have been widely validated for their effectiveness in compiler testing. JavaTailor [93] gets test programs by weaving ingredients extracted from JVM historical bug-revealing test programs for covering more JVM behaviors/paths. Orion [65] utilizes historical bug reports to inform the creation of test inputs for deep learning libraries. JOpFuzzer [32] is a JVM testing approach with a specific focusing on JIT bugs. JOpFuzzer searches for diverse configurations along the new dimension of optimization options. And JOpFuzzer learns the correlations between various code features and different optimization options based on rules to guide the process of seed mutation and option exploration. Compared to JOpFuzzer, OptFuzz utilizes LLMs for generating compilation options and is primarily dedicated to C/C++ compilers. Moreever, in theory, OptFuzz can also be extended to other compilers.

6.2 LLMs for software testing

The emergence of LLMs significantly enhanced all methods of software testing [76, 85], which includes innovations in various fields such as specification generation based on source code, test case generation, error localization, program repair, etc. Titanfuzz proposed using LLMs to generate code snippets directly and synthesize Python test cases for deep learning libraries [13]. FuzzGPT fine-tunes LLMs by collecting historical code segments that triggered errors in deep learning libraries and uses text prompts to generate Python test cases [14]. KernalGPT attempts to leverage LLM for enhancing kernel fuzzing [86]. Fuzz4all adopt carefully designed prompts to guide LLMs in generating test cases [82]. HistFuzz [70] propose combining LLM with historically triggered code snippets to generate comprehensive test cases. Cummins et al. propose using LLMs for code optimization [11]. LLMs have a high degree of automation, significantly reducing human intervention and increasing testing efficiency. Due to the high understanding of code rules and human language by pre-trained models, researchers can perform many

Just Accepted

tasks previously done by humans with the help of LLMs.

7 Conclusion

This study introduces a novel approach that employs LLMs to generate sophisticated compilation options for testing compilers. By integrating an IR-based code abstraction strategy, the efficiency of LLM generation is significantly enhanced. Additionally, this research has explored the utilization of LLMs to process bug reports for obtaining test programs, achieving promising results. We implement our approach as a prototype called OptFuzz. Through an extensive evaluation, we demonstrate the practical value of OptFuzz, as it successfully detected a total of 64 bugs for GCC and LLVM. Notably, 53 of these bugs have been confirmed, and 24 have been fixed by the developers, underscoring the significant impact and efficacy of OptFuzz in identifying compiler bugs. Also, the prototype of OptFuzz serves as a valuable resource for further research and development in compiler testing methodologies. To support reproducibility and follow-up research, we have made the OptFuzz artifact publicly available at https://doi.org/10.5281/zenodo.18960140.

Acknowledgments

We thank the anonymous reviewers and editors for their constructive feedback and careful guidance, which substantially improved the quality and clarity of this work. We are also grateful to the GCC and LLVM developers. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62272214), the Jiangsu Natural Science Foundation (Grant BK20231402), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), the "111 Center"(No. B26023), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] Andrew W. Appel and Maia Ginsburg. 1998. *Modern Compiler Implementation in C.* Press Syndicate of the University of Cambridge.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, and Jared et al. Kaplan. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[3] Alexander E. I. Brownlee, James Callan, Karine Even-Mendoza, and Alina Geiger et al. 2023. Enhancing Genetic Improvement Mutations Using Large Language Models. In *Search-Based Software Engineering - 15th International Symposium, SSBSE 2023, San Francisco, CA, USA, December 8, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 14415)*, Paolo Arcaini, Tao Yue, and Erik M. Fredericks (Eds.). Springer, 153–159. doi:10.1007/978-3-031-48796-5_13

[4] Junjie Chen, Yanwei Bai, Dan Hao, Yingfei Xiong, Hongyu Zhang, and Bing Xie. 2017. Learning to prioritize test programs for compiler testing. In *Proceedings of the 39th International Conference on Software Engineering* (Buenos Aires, Argentina) *(ICSE '17)*. IEEE Press, 700–711. doi:10.1109/ICSE.2017.70

[5] Junjie Chen, Wenxiang Hu, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2016. An empirical comparison of compiler testing techniques. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) *(ICSE '16)*. New York, NY, USA, 180–190. doi:10.1145/2884781.2884878

[6] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A Survey of Compiler Testing. *ACM Comput. Surv.* 53, 1 (2020), 4:1–4:36. doi:10.1145/3363562

[7] Junjie Chen and Chenyao Suo. 2022. Boosting Compiler Testing via Compiler Optimization Exploration. *ACM Trans. Softw. Eng. Methodol.* 31, 4, Article 72 (aug 2022), 33 pages. doi:10.1145/3508362

[8] Junjie Chen, Chenyao Suo, Jiajun Jiang, Peiqi Chen, and Xingjian Li. 2023. Compiler Test-Program Generation via Memoized Configuration Search. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2035–2047. doi:10.1109/ICSE48619.2023.00172

[9] Junjie Chen, Guancheng Wang, Dan Hao, Yingfei Xiong, Hongyu Zhang, and Lu Zhang. 2020. History-guided configuration diversification for compiler test-program generation. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering* (San Diego, California) *(ASE '19)*. IEEE Press, 305–316. doi:10.1109/ASE.2019.00037

[10] Shafiul Azam Chowdhury, Sohil Lal Shrestha, Taylor T. Johnson, and Christoph Csallner. 2020. SLEMI: equivalence modulo input (EMI) based mutation of CPS models for finding compiler bugs in Simulink. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) *(ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 335–346. doi:10.1145/3377811.3380381

Just Accepted

[11] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. 2023. Large Language Models for Compiler Optimization. arXiv:2309.07062 [cs.PL] https://arxiv.org/abs/2309.07062

[12] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, and Runxin Xu et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

[13] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 423–435. doi:10.1145/3597926.3598067

[14] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large Language Models are Edge-Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. New York, NY, USA, Article 70, 13 pages. doi:10.1145/3597503.3623343

[15] Alex Denisov and Stanislav Pankevich. 2018. Mull It Over: Mutation Testing Based on LLVM. In *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 25–31. doi:10.1109/ICSTW.2018.00024

[16] Vijay D'Silva, Mathias Payer, and Dawn Song. 2015. The Correctness-Security Gap in Compiler Optimization. In *Proceedings of the 2015 IEEE Security and Privacy Workshops (SPW '15)*. IEEE Computer Society, USA, 73–87.

[17] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General Language Model Pretraining with Autoregressive Blank Infilling. arXiv:2103.10360 [cs.CL] https://arxiv.org/abs/2103.10360

[18] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. arXiv:2204.05999 [cs.SE] https://arxiv.org/abs/2204.05999

[19] Hongyan Gao, Yibiao Yang, Maolin Sun, Jiangchang Wu, Yuming Zhou, and Baowen Xu. 2025. Clozemaster: Fuzzing Rust Compiler by Harnessing Llms for Infilling Masked Real Programs . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1422–1435. doi:10.1109/ICSE55347.2025.00175

[20] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R. Lyu. 2023. What Makes Good In-Context Demonstrations for Code Intelligence Tasks with LLMs?. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 761–773. doi:10.1109/ASE56229.2023.00109

[21] GCC developers. 2024. GCC Bugzilla. (2024). https://gcc.gnu.org/bugzilla/ Accessed 24 July 2024..

[22] GCC developers. 2024. GCC Command Options. (2024). https://gcc.gnu.org/onlinedocs/gcc/Invoking-GCC.html Accessed 20 June 2024..

[23] GCC developers. 2024. GCC GIMPLE. (2024). https://gcc.gnu.org/onlinedocs/gccint/GIMPLE.html Accessed 24 July 2024..

[24] GCC developers. 2024. GCC Passes. (2024). https://gcc.gnu.org/onlinedocs/gcc-3.3.6/gccint/Passes.html Accessed 24 July 2024..

[25] GCC developers. 2024. GCC RTL. (2024). https://gcc.gnu.org/onlinedocs/gccint/RTL.html Accessed 24 July 2024..

[26] GCC developers. 2024. Installing GCC: Testing. (2024). https://gcc.gnu.org/install/test.html Accessed 20 June 2024..

[27] GCC developers. 2024. Options That Control Optimization. (2024). https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html Accessed 20 June 2024..

[28] Qiuhan Gu. 2023. LLM-Based Code Generation Method for Golang Compiler Testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. New York, NY, USA, 2201–2203. doi:10.1145/3611643.3617850

[29] Kenneth Hoste and Lieven Eeckhout. 2008. Cole: compiler optimization level exploration. In *Proceedings of the 6th Annual IEEE/ACM International Symposium on Code Generation and Optimization* (Boston, MA, USA) *(CGO '08)*. New York, NY, USA, 165–174. doi:10.1145/1356058.1356080

[30] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An Empirical Study on Fine-Tuning Large Language Models of Code for Automated Program Repair. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1162–1174. doi:10.1109/ASE56229.2023.00181

[31] Li Huang, Weifeng Sun, Meng Yan, Zhongxin Liu, Yan Lei, and David Lo. 2025. Neuron Semantic-Guided Test Generation for Deep Neural Networks Fuzzing. *ACM Trans. Softw. Eng. Methodol.* 34, 1, Article 15 (Dec. 2025), 38 pages. doi:10.1145/3688835

[32] Haoxiang Jia, Ming Wen, Zifan Xie, Xiaochen Guo, Rongxin Wu, Maolin Sun, Kang Chen, and Hai Jin. 2023. Detecting JVM JIT Compiler Bugs via Exploring Two-Dimensional Input Spaces. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 43–55. doi:10.1109/ICSE48619.2023.00016

[33] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678. doi:10.1109/TSE.2010.62

[34] He Jiang, Zhide Zhou, Zhilei Ren, Jingxuan Zhang, and Xiaochen Li. 2021. CTOS: Compiler Testing for Optimization Sequences of LLVM. *IEEE Transactions on Software Engineering* PP, 99 (2021), 1–1.

[35] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2023. Explainable Automated Debugging via Large Language Model-driven Scientific Debugging. *arXiv preprint arXiv:2304.02195* (2023).

Just Accepted

[36] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) *(PLDI '14)*. New York, NY, USA, 216–226. doi:10.1145/2594291.2594334

[37] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Finding deep compiler bugs via guided stochastic program mutation. *SIGPLAN Not.* 50, 10 (oct 2015), 386–399. doi:10.1145/2858965.2814319

[38] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Randomized Stress-Testing of Link-Time Optimizers. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis* (Baltimore, MD, USA) *(ISSTA 2015)*. Association for Computing Machinery, New York, NY, USA, 327–337. doi:10.1145/2771783.2771785

[39] Ahcheong Lee, Youngseok Choi, Shin Hong, Yunho Kim, Kyutae Cho, and Moonzoo Kim. 2025. ZigZagFuzz: Interleaved Fuzzing of Program Options and Files. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 39 (Jan. 2025), 31 pages. doi:10.1145/3697014

[40] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 66–79. doi:10.1145/3600006.3613140

[41] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting Compiler Testing by Injecting Real-World Code. *Proc. ACM Program. Lang.* 8, PLDI, Article 156 (June 2024), 23 pages. doi:10.1145/3656386

[42] Yuanbo Li, Shuo Ding, Qirun Zhang, and Davide Italiano. 2020. Debug information validation for optimized code. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. New York, NY, USA, 1052–1065. doi:10.1145/3385412.3386020

[43] Christopher Lidbury, Andrei Lascu, Nathan Chong, and Alastair F. Donaldson. 2015. Many-core compiler fuzzing. *SIGPLAN Not.* 50, 6 (jun 2015), 65–76. doi:10.1145/2813885.2737986

[44] Jiawei Liu, Yuxiang Wei, Sen Yang, Yinlin Deng, and Lingming Zhang. 2022. Coverage-guided tensor compiler fuzzing with joint IR-pass mutation. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 73 (apr 2022), 26 pages. doi:10.1145/3527317

[45] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random Testing for C and C++ Compilers with YARPGen. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 196 (Nov. 2020), 25 pages. doi:10.1145/3428264

[46] Livinskii, Vsevolod and Babokin, Dmitry and Regehr, John. 2023. Fuzzing Loop Optimizations in Compilers for C++ and Data-Parallel Languages. *Proc. ACM Program. Lang.* 7, PLDI, Article 181 (jun 2023), 22 pages. doi:10.1145/3591295

[47] LLVM developers. 2024. LLVM Bugzilla. (2024). https://bugs.llvm.org Accessed 24 July 2024..

[48] LLVM developers. 2024. LLVM Compilation options. (2024). https://clang.llvm.org/docs/ClangCommandLineReference.html#compilation-options Accessed 20 June 2024..

[49] LLVM developers. 2024. LLVM Debian/Ubuntu nightly packages. (2024). https://apt.llvm.org Accessed 20 June 2024..

[50] LLVM developers. 2024. LLVM Github Issues. (2024). https://github.com/llvm/llvm-project/issues Accessed 24 July 2024..

[51] LLVM developers. 2024. LLVM IR. (2024). https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl03.html Accessed 24 July 2024..

[52] LLVM Developers. 2024. LLVM Issue 95366. https://github.com/llvm/llvm-project/issues/95366

[53] LLVM Developers. 2024. LLVM Issue 95495. https://github.com/llvm/llvm-project/issues/95495

[54] LLVM developers. 2024. LLVM Pass Manager. (2024). https://llvm.org/docs/NewPassManager.html Accessed 24 July 2024..

[55] LLVM developers. 2024. Test-suite Guide for LLVM. (2024). https://llvm.org/docs/TestSuiteGuide.html Accessed 20 June 2024..

[56] LLVM Developers. 2024. Writing an LLVM Pass. https://llvm.org/docs/WritingAnLLVMPass.html

[57] Matt Godbolt. 2024. Compiler Explorer. https://godbolt.org/

[58] Sania Nayab, Giulio Rossolini, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. 2024. Concise Thoughts: Impact of Output Length on LLM Reasoning and Cost. arXiv:2407.19825 [cs.CL] https://arxiv.org/abs/2407.19825

[59] OpenAI, Josh Achiam, Steven Adler, and Sandhini Agarwal et. al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[60] Yun Peng, Shuzheng Gao, Cuiyun Gao, Yintong Huo, and Michael Lyu. 2024. Domain Knowledge Matters: Improving Prompts with Fix Templates for Repairing Python Type Errors. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. New York, NY, USA, Article 4, 13 pages. doi:10.1145/3597503.3608132

[61] Laura Plein, Wendkûuni C. Ouédraogo, Jacques Klein, and Tegawendé F. Bissyandé. 2024. Automatic Generation of Test Cases based on Bug Reports: a Feasibility Study with Large Language Models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (Lisbon, Portugal) *(ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, USA, 360–361. doi:10.1145/3639478.3643119

[62] Ruixiang Qian, Quanjun Zhang, Chunrong Fang, Ding Yang, Shun Li, Binyu Li, and Zhenyu Chen. 2025. DiPri: Distance-Based Seed Prioritization for Greybox Fuzzing—RCR Report. *ACM Trans. Softw. Eng. Methodol.* 34, 1, Article 27 (Jan. 2025), 13 pages. doi:10.1145/3701298

[63] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) *(PLDI '12)*.

Just Accepted

Association for Computing Machinery, New York, NY, USA, 335–346. doi:10.1145/2254064.2254104

[64] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] https://arxiv.org/abs/2308.12950

[65] Nima Shiri Harzevili, Mohammad Mahdi Mohajer, Moshi Wei, Hung Viet Pham, and Song Wang. 2024. History-Driven Fuzzing For Deep Learning Libraries. *ACM Trans. Softw. Eng. Methodol.* (Aug. 2024). doi:10.1145/3688838

[66] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding and Analyzing Compiler Warning Defects. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) *(ICSE '16)*. Association for Computing Machinery, New York, NY, USA, 203–213. doi:10.1145/2884781.2884879

[67] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Amsterdam, Netherlands) *(OOPSLA 2016)*. Association for Computing Machinery, New York, NY, USA, 849–863. doi:10.1145/2983990.2984038

[68] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. 2016. Toward understanding compiler bugs in GCC and LLVM. In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) *(ISSTA 2016)*. Association for Computing Machinery, New York, NY, USA, 294–305. doi:10.1145/2931037.2931074

[69] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. 2018. Perses: syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 361–371. doi:10.1145/3180155.3180236

[70] Maolin Sun, Yibiao Yang, Ming Wen, Yongcong Wang, Yuming Zhou, and Hai Jin. 2023. Validating SMT Solvers via Skeleton Enumeration Empowered by Historical Bug-Triggering Inputs. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 69–81. doi:10.1109/ICSE48619.2023.00018

[71] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul R. Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, and Clemens Meyer. 2024. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805 [cs.CL] https://arxiv.org/abs/2312.11805

[72] Theodoros Theodoridis, Manuel Rigger, and Zhendong Su. 2022. Finding missed optimizations through the lens of dead code elimination. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 697–709. doi:10.1145/3503222.3507764

[73] Spyridon Triantafyllis, Manish Vachharajani, Neil Vachharajani, and David I. August. 2003. Compiler optimization-space exploration. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (San Francisco, California, USA) *(CGO '03)*. IEEE Computer Society, USA, 204–215.

[74] Christos Tsigkanos, Pooja Rani, Sebastian Müller, and Timo Kehrer. 2023. Large Language Models: The Next Frontier for Variable Discovery within Metamorphic Testing?. In *IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21-24, 2023*, Tao Zhang, Xin Xia, and Nicole Novielli (Eds.). IEEE, 678–682. doi:10.1109/SANER56733.2023.00070

[75] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[76] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* PP (04 2024), 1–27. doi:10.1109/TSE.2024.3368208

[77] Taiyan Wang, Ruipeng Wang, Yu Chen, Lu Yu, Zulie Pan, Min Zhang, Huimin Ma, and Jinghua Zheng. 2024. Enhancing Black-box Compiler Option Fuzzing with LLM through Command Feedback. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. 319–330. doi:10.1109/ISSRE62328.2024.00039

[78] Theodore Luo Wang, Yongqiang Tian, Yiwen Dong, Zhenyang Xu, and Chengnian Sun. 2023. Compilation Consistency Modulo Debug Information. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 146–158. doi:10.1145/3575693.3575740

[79] Weishi Wang, Yue Wang, Shafiq Joty, and Steven C. H. Hoi. 2023. RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 146–158. doi:10.1145/3611643.3616256

[80] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe,

Just Accepted

and Paolo Tonella (Eds.). ACM, 172–184. doi:10.1145/3611643.3616271

[81] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large Language Models in Fault Localisation. *CoRR* abs/2308.15276 (2023). doi:10.48550/ARXIV.2308.15276 arXiv:2308.15276

[82] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 126, 13 pages. doi:10.1145/3597503.3639121

[83] Jianhao Xu, Luca Di Bartolomeo, Flavio Toffalini, Bing Mao, and Mathias Payer. 2023. WarpAttack: Bypassing CFI through Compiler-Introduced Double-Fetches. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1271–1288. doi:10.1109/SP46215.2023.10179433

[84] Jianhao Xu, Kangjie Lu, Zhengjie Du, Zhu Ding, Linke Li, Qiushi Wu, Mathias Payer, and Bing Mao. 2023. Silent bugs matter: a study of compiler-introduced security bugs. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) *(SEC '23)*. USENIX Association, USA, Article 205, 18 pages.

[85] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2023. White-box compiler fuzzing empowered by large language models. *arXiv preprint arXiv:2310.15991* (2023).

[86] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2023. KernelGPT: Enhanced Kernel Fuzzing via Large Language Models. arXiv:2401.00563 [cs.CR] https://arxiv.org/abs/2401.00563

[87] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. New York, NY, USA, 283–294. doi:10.1145/1993498.1993532

[88] Yibiao Yang, Yanyan Jiang, Zhiqiang Zuo, Yang Wang, Hao Sun, Hongmin Lu, Yuming Zhou, and Baowen Xu. 2020. Automatic self-validation for code coverage profilers. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering* (San Diego, California) *(ASE '19)*. IEEE Press, 79–90. doi:10.1109/ASE.2019.00018

[89] Cen Zhang, Mingqiang Bai, Yaowen Zheng, Yeting Li, Xiaofei Xie, Yuekang Li, Wei Ma, Limin Sun, and Yang Liu. 2023. Understanding Large Language Model Based Fuzz Driver Generation. *CoRR* abs/2307.12469 (2023). doi:10.48550/ARXIV.2307.12469 arXiv:2307.12469

[90] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal program enumeration for rigorous compiler testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) *(PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 347–361. doi:10.1145/3062341.3062379

[91] Zenong Zhang, George Klees, Eric Wang, Michael Hicks, and Shiyi Wei. 2023. Fuzzing Configurations of Program Options. *ACM Trans. Softw. Eng. Methodol.* 32, 2, Article 53 (mar 2023), 21 pages. doi:10.1145/3580597

[92] Yingquan Zhao, Junjie Chen, Ruifeng Fu, Haojie Ye, and Zan Wang. 2023. Testing the Compiler for a New-Born Programming Language: An Industrial Case Study (Experience Paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. New York, NY, USA, 551–563. doi:10.1145/3597926.3598077

[93] Yingquan Zhao, Zan Wang, Junjie Chen, Mengdi Liu, Mingyuan Wu, Yuqun Zhang, and Lingming Zhang. 2022. History-driven test program synthesis for JVM testing. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. New York, NY, USA, 1133–1144. doi:10.1145/3510003.3510059

[94] Hao Zhong. 2023. Enriching Compiler Testing with Real Program from Bug Report. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (Rochester, MI, USA) *(ASE '22)*. New York, NY, USA, Article 40, 12 pages. doi:10.1145/3551349.3556894

[95] Mingxuan Zhu and Dan Hao. 2023. Compiler Auto-Tuning via Critical Flag Selection. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1000–1011. doi:10.1109/ASE56229.2023.00209

Just Accepted